

推荐算法



时间：2025年5月

TRANSWARP
星环科技



目录 >

CONTENTS

- 1 推荐系统概述
- 2 基于内存的协同过滤
- 3 基于模型的协同过滤
- 4 关联规则模型

The background features a light gray network graph with nodes and edges. Two large, semi-transparent spheres are positioned on the left and right sides, each containing a dense network of nodes and edges. In the center, there is a dark blue circle containing the number '1' and the word 'chapter' below it.

1 chapter

推荐系统概述

推荐系统的应用

由于推荐系统和商业的联系是如此紧密，从1992年到2020年，经过28年的时间，其使用场景拓展到了互联网应用的方方面面，将推荐系统发挥到极致的、最典型的公司莫过于**字节跳动**，推荐系统作为其最核心的底层服务支撑起了其5000亿人民币的市值。



商品

新闻

短视频

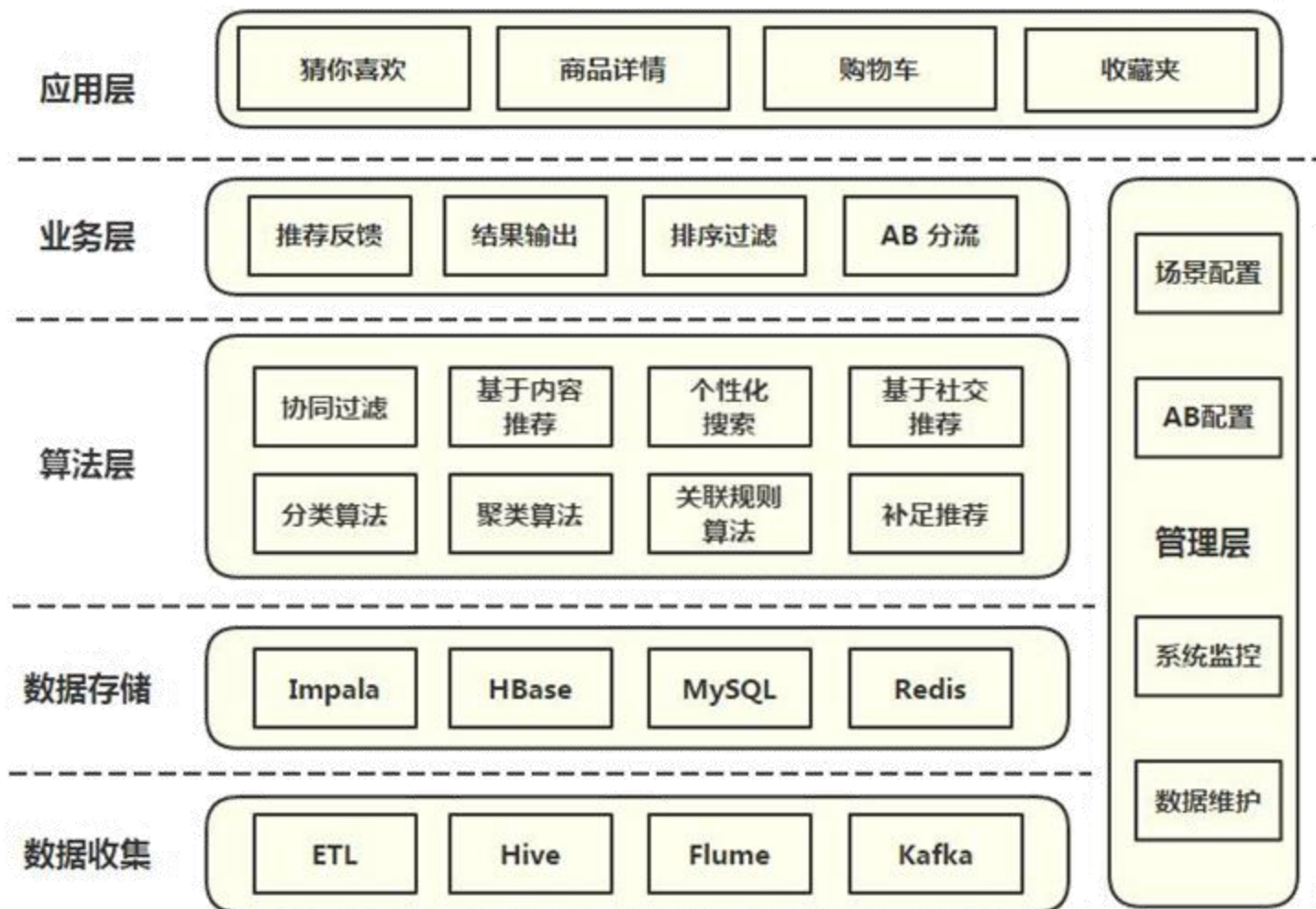
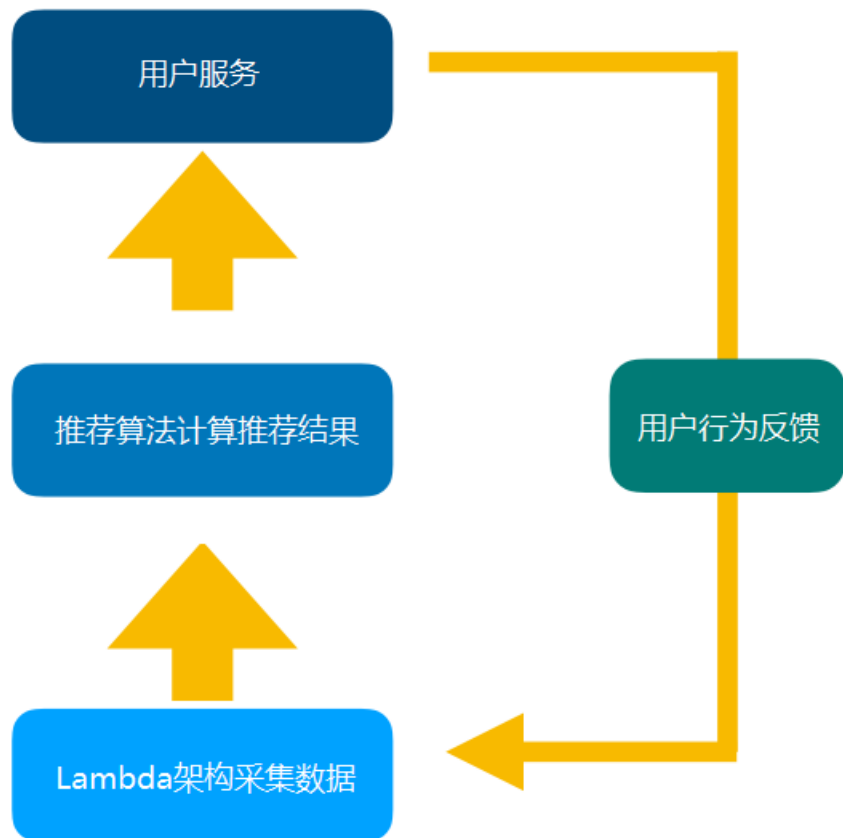
酒店景点

人

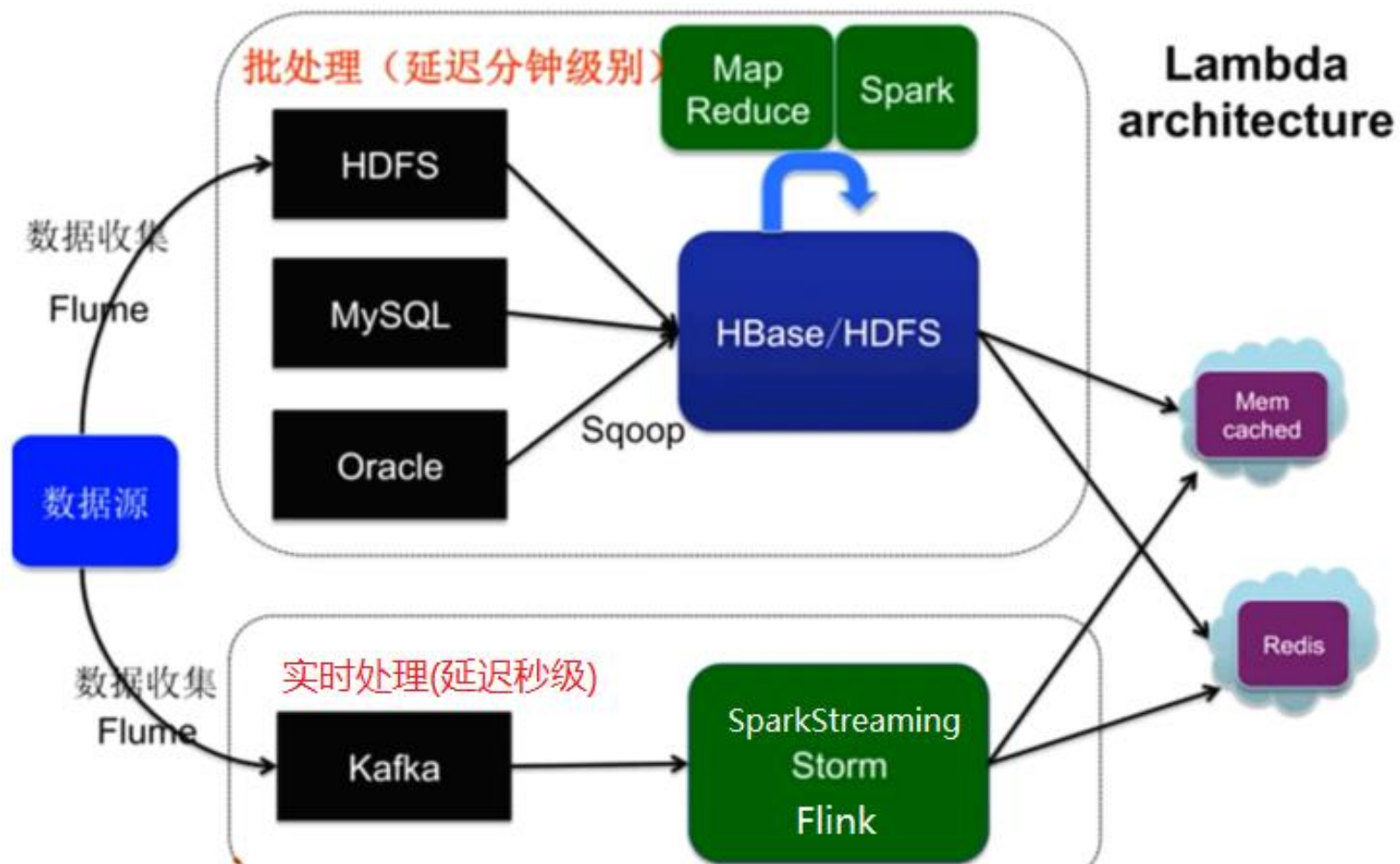
推荐系统的应用



推荐系统架构



推荐系统架构



推荐系统架构

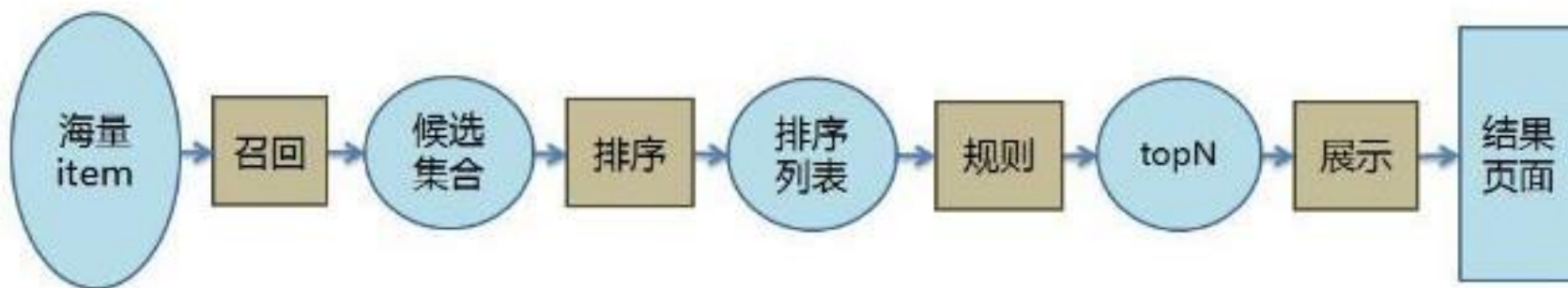
■ 召回阶段 (海选)

- 召回决定了最终推荐结果的天花板
- 常用算法：协同过滤、基于内容

■ 排序阶段 (精选)

- 召回决定了最终推荐结果的天花板, 排序逼近这个极限, 决定了最终的推荐效果
- CTR预估 (点击率预估 使用LR算法) 估计用户是否会点击某个商品 需要用户的点击数据

■ 干预：过滤、量控、打散



推荐算法分类

■ 非个性化推荐:

- 基于规则的推荐, top10热榜,大家都在看等
- 基于人口属性的推荐, 湖南人推荐辣的食物等

■ 个性化推荐: 基于用户对商品的偏好信息来进行其它商品的推荐

- 基于内容的推荐: 根据用户历史浏览信息, 推荐相似物品
- 基于**内存的协同过滤**: **userCF**、**itemCF**
- 基于**模型的协同过滤**: Baseline、**SVD**、FM、FFM, DeepFM、图、深度学习
- 基于知识的推荐: 利用知识图谱进行推荐
- 基于**关联规则**的推荐: 买了A的人也买了B

1
2
chapter

基于内存的协同过滤

协同过滤推荐算法概述

■ Memory-Based CF

- 利用用户行为数据（点击，购买，评分）计算相似度（**余弦，皮尔逊，杰卡德**)
 - 计算**用户**之间的**相似度**：**基于用户**的协同过滤
 - 计算**物品**之间的**相似度**：**基于物品**的协同过滤
- 利用**最相似其它用户/最相似其它物品**产生推荐结果

■ Model-Based CF

- 利用**机器学习算法**预测用户对物品的喜好程度
- 当用户-物品的评分数据比较稀疏的时候更适合使用Model-Based CF（基于模型的协同过滤）

协同过滤推荐算法计算目标

- **输入**：稀疏的用户-物品评分矩阵
- **输出**：预测的**完整**的用户-物品评分矩阵

						
	4	3			5	
	5		4		4	
	4		5	3	4	
		3				5
		4				4
			2	4		5

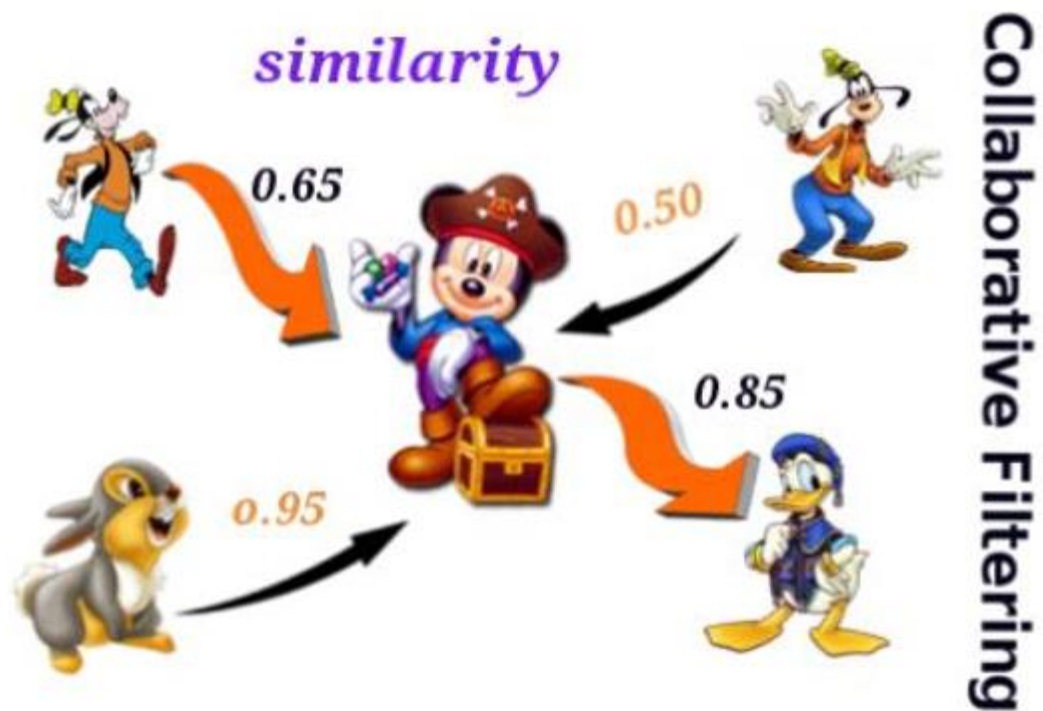


						
	4	3	3	4	5	2
	5	3	4	3	4	4
	4	2	5	3	4	2
	2	3	3	4	2	5
	4	4	3	3	4	4
	3	3	2	4	3	5

基于内存的协同过滤

- 协同过滤推荐算法基于以下假设：
 - “跟你喜好**相似的人**喜欢的东西你也很有可能喜欢”：基于用户的协同过滤推荐 (**User-based CF**)
 - “跟你喜欢的东西**相似的东西**你也很有可能喜欢”：基于物品的协同过滤推荐 (**Item-based CF**)
- 计算步骤：
 - 计算出**相似度矩阵**得到**TOP-N相似**的人或物品
 - 根据**相似的人或物品**产生推荐结果

User-based CF



User-based CF案例

■ 1、计算用户相似度矩阵

						
	4	3			5	
	5		4		4	
	4		5	3	4	
		3				5
		4				4
			2	4		5



						
	1.00	0.75	0.63	0.22	0.30	0.00
	0.75	1.00	0.91	0.00	0.00	0.16
	0.63	0.91	1.00	0.00	0.00	0.40
	0.22	0.00	0.00	1.00	0.97	0.64
	0.30	0.00	0.00	0.97	1.00	0.53
	0.00	0.16	0.40	0.64	0.53	1.00

User-based CF案例

■ 2、选取topN相似用户进行推荐

	4	3			5	
	5		4		4	
	4		5	3	4	
		3				5
		4				4
			2	4		5

	1.00	0.75	0.63	0.22	0.30	0.00
	0.75	1.00	0.91	0.00	0.00	0.16
	0.63	0.91	1.00	0.00	0.00	0.40
	0.22	0.00	0.00	1.00	0.97	0.64
	0.30	0.00	0.00	0.97	1.00	0.53
	0.00	0.16	0.40	0.64	0.53	1.00



$$\hat{r}_{ui} = \frac{\sum_{v \in N_i^k(u)} sim(u, v) * r_{vi}}{\sum_{v \in N_i^k(u)} sim(u, v)}$$

(0.7 x) + (0.6 x) =

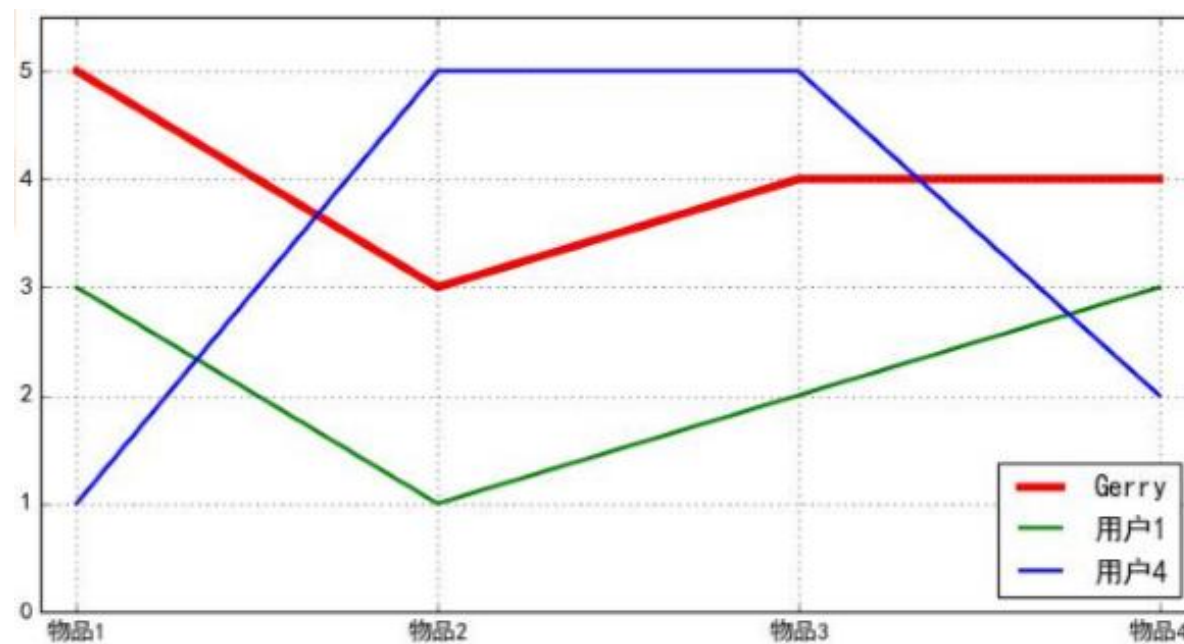
already rated by user
 $(0.7 \times 4 + 0.6 \times 5) / (0.7 + 0.6) = 4.5$
 $(0.6 \times 3) / 0.6 = 3.0$
 already rated by user

图4：为一个用户进行推荐。我们将选取最相似的两个用户所评价的书，进行加权，然后推荐加权分数最高且目标用户未评价过的图书

存在的问题？

■ 某些用户打分偏高（低）怎么办？

	物品1	物品2	物品3	物品4	物品5
Gerry	5	3	4	4	?
用户1	3	1	2	3	3
用户2	4	3	4	3	5
用户3	3	3	3	5	4
用户4	1	5	2	4	1

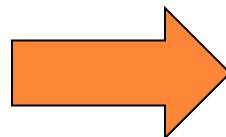


均值转换

- 计算每个用户打分的平均分，使用**分值-平均分**作为建模数据

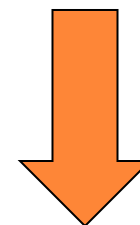
	物品1	物品2	物品3	物品4	物品5
Gerry	5	3	4	4	?
用户1	3	1	2	3	3
用户2	4	3	4	3	5
用户3	3	3	3	5	4
用户4	1	5	2	4	1

去均值



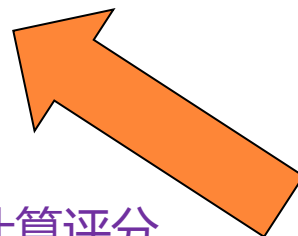
	物品1	物品2	物品3	物品4	物品5	均值
Gerry	1	-1	0	0	?	4
用户1	0.6	-1.4	-0.4	0.6	0.6	2.4
用户2	0.2	-0.8	0.2	-0.8	1.2	3.8
用户3	-0.6	-0.6	-0.6	1.4	0.4	3.6
用户4	-1.6	2.4	-0.6	1.4	-1.6	2.6

计算相似度



	用户1	用户2	用户3	用户4
Gerry	0.839	0.707	0.0	-0.894

计算评分



$$\text{pred}(\text{gerry}, \text{product5}) = 4 + \frac{0.853 * (3 - 2.4) + 0.707 * (5 - 3.8)}{0.853 + 0.707} = 4.872$$

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_i^k(u)} \text{sim}(u, v) * (r_{vi} - \bar{r}_v)}{\sum_{v \in N_i^k(u)} \text{sim}(u, v)}$$

Item-based CF案例

■ 1、计算物品相似度矩阵

						
	4	3			5	
	5		4		4	
	4		5	3	4	
		3				5
		4				4
			2	4		5



						
	1.00	0.27	0.79	0.32	0.98	0.00
	0.27	1.00	0.00	0.00	0.34	0.65
	0.79	0.00	1.00	0.69	0.71	0.18
	0.32	0.00	0.69	1.00	0.32	0.49
	0.98	0.34	0.71	0.32	1.00	0.00
	0.00	0.65	0.18	0.49	0.00	1.00

Item-based CF案例

■ 2、选取topN相似物品进行推荐

	4	3			5	
	5		4		4	
	4		5	3	4	
		3				5
		4				4
			2	4		5

	1.00	0.27	0.79	0.32	0.98	0.00
	0.27	1.00	0.00	0.00	0.34	0.65
	0.79	0.00	1.00	0.69	0.71	0.18
	0.32	0.00	0.69	1.00	0.32	0.49
	0.98	0.34	0.71	0.32	1.00	0.00
	0.00	0.65	0.18	0.49	0.00	1.00



这两本与黑色这本最相似

$$\hat{r}_{ui} = \frac{\sum_{j \in N_u^k(i)} sim(i, j) * r_{uj}}{\sum_{j \in N_u^k(i)} sim(i, j)}$$

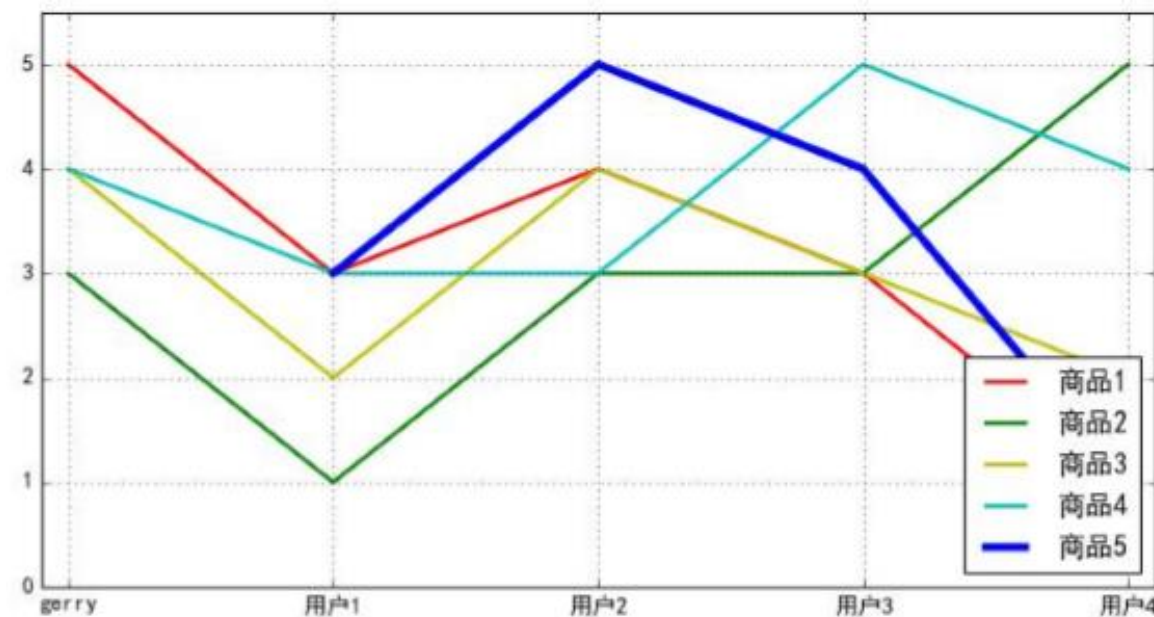
already rated by user
 $(0.8 \times 4 + 0.7 \times 5) / (0.8 + 0.7) = 4.5$
 already rated by user
 $(0.7 \times 3) / 0.7 = 3.0$

图7：为用户进行推荐。我们选取他们评价过的图书，找出与他们最相似的前两本书，进行加权，然后推荐给用户加权分最高且他没有读过的书

同样存在的问题？

■ 某些物品打分偏高（低）怎么办？

	物品1	物品2	物品3	物品4	物品5
Gerry	5	3	4	4	?
用户1	3	1	2	3	3
用户2	4	3	4	3	5
用户3	3	3	3	5	4
用户4	1	5	2	4	1

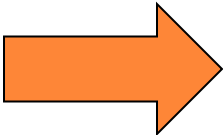


均值转换

- 计算**每个物品**打分的**平均分**，使用**分值-平均分**作为建模数据

$$\hat{r}_{ui} = \frac{\sum_{j \in N_u^k(i)} \text{sim}(i, j) * r_{uj}}{\sum_{j \in N_u^k(i)} \text{sim}(i, j)}$$

去均值


$$\hat{r}_{ui} = \bar{r}_i + \frac{\sum_{j \in N_u^k(i)} \text{sim}(i, j) * (r_{uj} - \bar{r}_j)}{\sum_{j \in N_u^k(i)} \text{sim}(i, j)}$$

导包和数据

```
1 import pandas as pd
2 import numpy as np
3
4 users = ["User1", "User2", "User3", "User4", "User5"]
5 items = ["Item A", "Item B", "Item C", "Item D", "Item E"]
6 #用户购买记录数据集
7 datasets = [
8     [5, 3, 4, 4, None],
9     [3, 1, 2, 3, 3],
10    [4, 3, 4, 3, 5],
11    [3, 3, 1, 5, 4],
12    [1, 5, 5, 2, 1],
13 ]
```

计算相似度矩阵

```
1 df = pd.DataFrame(datasets,
2                     columns=items,
3                     index=users)
4 print("用户之间的两两相似度:")
5 # 直接计算皮尔逊相关系数
6 # 默认是按列进行计算, 因此如果计算用户间的相似度, 当前需要进行转置
7 user_similar = df.T.corr()
8 print(user_similar.round(4))
9
10 print("物品之间的两两相似度:")
11 item_similar = df.corr()
12 print(item_similar.round(4))
```

用户之间的两两相似度:

	User1	User2	User3	User4	User5
User1	1.0000	0.8528	0.7071	0.0000	-0.7921
User2	0.8528	1.0000	0.4677	0.4900	-0.9001
User3	0.7071	0.4677	1.0000	-0.1612	-0.4666
User4	0.0000	0.4900	-0.1612	1.0000	-0.6415
User5	-0.7921	-0.9001	-0.4666	-0.6415	1.0000

物品之间的两两相似度:

	Item A	Item B	Item C	Item D	Item E
Item A	1.0000	-0.4767	-0.1231	0.5322	0.9695
Item B	-0.4767	1.0000	0.6455	-0.3101	-0.4781
Item C	-0.1231	0.6455	1.0000	-0.7206	-0.4276
Item D	0.5322	-0.3101	-0.7206	1.0000	0.5817
Item E	0.9695	-0.4781	-0.4276	0.5817	1.0000

User-Based CF 评分预测

```
1  # User-Based CF 评分预测: 使用用户间的相似度进行预测
2  pred_user = user_similar.loc[:, 'User1']
3  sum1 = 0
4  sum2 = 0
5
6  for i in range(5):
7      if pred_user[i] > 0:
8          if pred_user[i] != 1:
9              sum1 += pred_user[i].round(2)*df.iloc[i, 4]
10             sum2 += pred_user[i].round(2)
11
12  pred1 = sum1/sum2
13  print(pred1.round(2))
```

3.91

Item-Based CF 评分预测

```
1  # Item-Based CF 评分预测: 使用物品间的相似度进行预测
2  pred_item = item_similar.loc[:, 'Item E']
3  sum3 = 0
4  sum4 = 0
5
6  for i in range(5):
7      if pred_item[i] > 0:
8          if pred_item[i] != 1:
9              sum3 += pred_item[i].round(2)*df.iloc[0,i]
10             sum4 += pred_item[i].round(2)
11
12  pred2 = sum3/sum4
13  print(pred2.round(2))
```

4.63

小结

本节我们主要学习了**User-based CF**和**Item-based CF**:

- **User-based**: 计算用户相似度
- **Item-based**: 计算物品相似度
- 最终目的: 计算未知的用户-物品打分
- 打分偏高问题: **均值转换**
- 使用场景:
 - 电商: 用户多于物品, 使用**Item-based**
 - 社交: 物品多于用户, 使用**User-based**



The background features a complex network diagram with numerous nodes (represented by small grey and black dots) connected by thin, light grey lines. These connections form a dense web that fills the entire frame, with some areas appearing more concentrated than others. The overall aesthetic is technical and modern.

1
3
chapter

基于模型的协同过滤

Model-Based 协同过滤算法简介

■ **Model-Based** 协同过滤算法会利用**用户-物品评分矩阵**训练**机器学习模型**，将**用户和物品的id**带入到模型中就会得到**用户对物品的评分预测**，Model-Based CF算法分类如下：

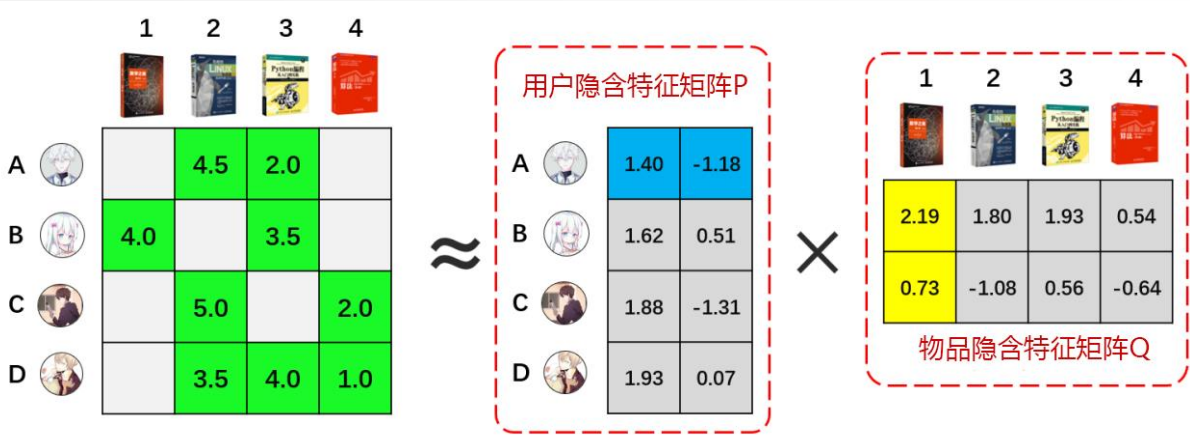
- 基于分类算法、回归算法、聚类算法
- 基于矩阵分解的推荐：**SVD、FunkSVD (LFM) 、BiasSvd**
- 基于神经网络算法
- 基于图模型算法

Model-Based 协同过滤算法简介

	iphone6	iphone6 plus
User-1	2.0	?s
User-2	?	2.4

	高端手机	大屏幕	性能	续航
iphone6	1	0.7	0.8	0.7
iphone6 plus	1	0.9	0.9	0.6

	高端手机	大屏幕	性能	续航
User-1	0.8	0.5	0.8	0.3
User-2	0.9	0.8	0.6	0.4



写出来是为了方便理解，事实上我们不知道，是**隐含**的语义

物品具备的喜好特征

用户的隐含喜好

Traditional SVD

- 通常SVD矩阵分解指的是SVD（奇异值）分解技术，其公式如下：

$$M_{m \times n} = U_{m \times k} \Sigma_{k \times k} V_{k \times n}^T$$

Approximation of M using k Factors

- SVD分解的前提：矩阵是**稠密**的,稀疏的情况下一般用**均值**填充

FunkSVD (LFM)

- Funk SVD不再将矩阵分解为3个矩阵，而是分解为2个矩阵(用户-隐含特征矩阵Q, 物品-隐含特征矩阵P)

	1	2	3	4
				
A 		4.5	2.0	
B 	4.0		3.5	
C 		5.0		2.0
D 		3.5	4.0	1.0

 $\approx$

用户隐含特征矩阵P	
A 	1.40 -1.18
B 	1.62 0.51
C 	1.88 -1.31
D 	1.93 0.07

 $\times$

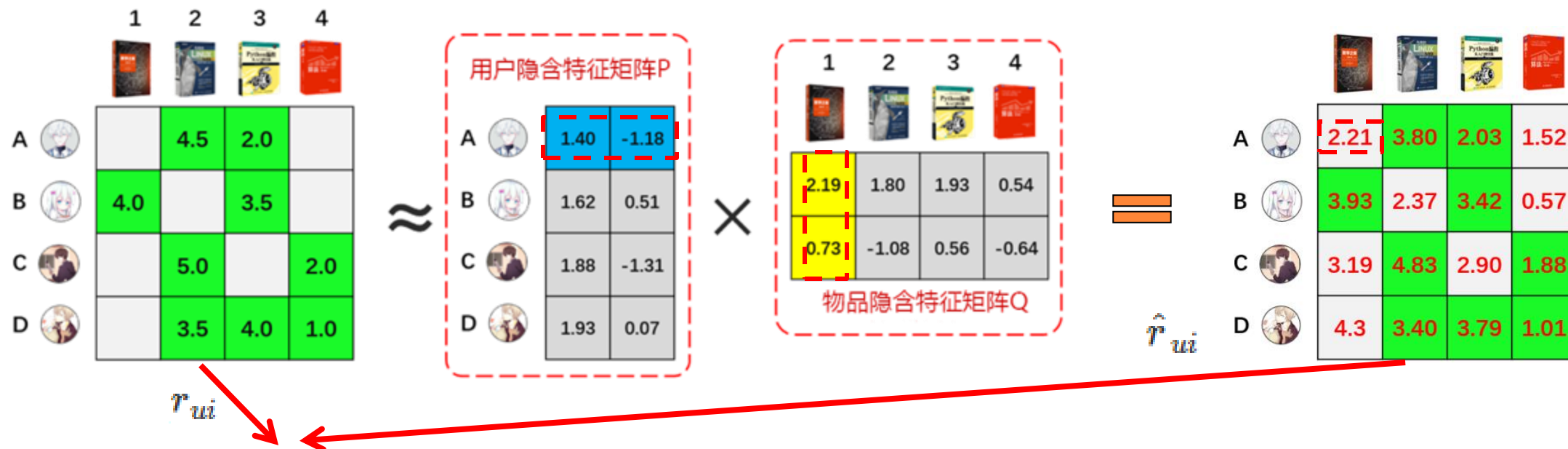
1	2	3	4
			
2.19	1.80	1.93	0.54
0.73	-1.08	0.56	-0.64

物品隐含特征矩阵Q

表示用户A对隐含特征1的权重值

表示隐含特征1在物品1上的权重值

FunkSVD (LFM)



$$Cost = \sum_{u,i \in R} (r_{ui} - \hat{r}_{ui})^2$$

$$= \sum_{u,i \in R} (r_{ui} - \sum_{k=1}^k p_{uk} q_{ik})^2$$

加正则

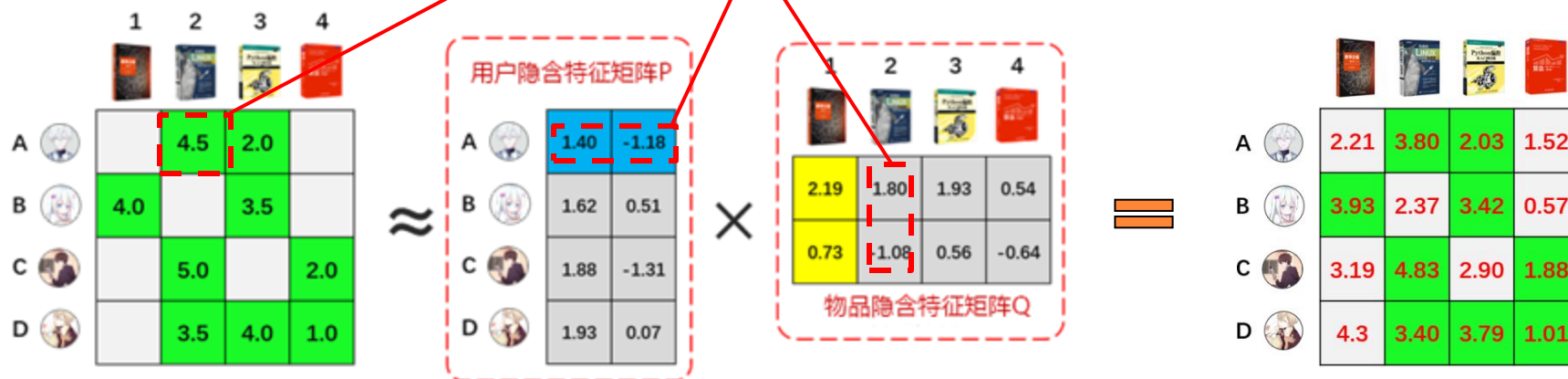
$$Cost = \sum_{u,i \in R} (r_{ui} - \sum_{k=1}^k p_{uk} q_{ik})^2 + \lambda (\sum_U p_{uk}^2 + \sum_I q_{ik}^2)$$

梯度下降求解

FunkSVD到BiasSVD:

$$\operatorname{argmin}_{p_j, q_j} \sum_{(i,j) \in K} (m_{ij} - q_j^T p_i)^2 + \lambda (\|p_i\|_2^2 + \|q_j\|_2^2)$$

FunkSVD



BiasSVD

$$\operatorname{argmin}_{p_i, q_j} \sum_{(i,j)} (m_{ij} - \mu - b_i - b_j - q_j^T p_i)^2 + \lambda (\|p_i\|_2^2 + \|q_j\|_2^2 + \|b_i\|_2^2 + \|b_j\|_2^2)$$



导入数据

```
import pandas as pd
import numpy as np
dtype = [("userId", np.int32), ("movieId", np.int32), ("rating", np.float32)]
dataset = pd.read_csv("data/ratings.csv", usecols=range(3), dtype=dict(dtype))
dataset.head()
```

	userId	movieId	rating
0	1	31	2.5
1	1	1029	3.0
2	1	1061	3.0
3	1	1129	2.0
4	1	1172	4.0

设置超参数

```
1 alpha = 0.02 # 学习率
2 reg_p = 0.01 # P矩阵正则
3 reg_q = 0.01 # Q矩阵正则
4 number_LatentFactors = 10 # 隐式类别数量
5 number_epochs = 100 # 最大迭代次数
6 columns = ['userId', 'movieId', 'rating']
```

构建数据

```
1 dataset = pd.DataFrame(dataset)
2
3 users_ratings = dataset.groupby(columns[0]).agg([list]) [[columns[1], columns[2]]]
4 items_ratings = dataset.groupby(columns[1]).agg([list]) [[columns[0], columns[2]]]
5
6 globalMean = dataset[columns[2]].mean() #全局平均值
```

```
1 # User-LF : 用户隐语义矩阵
2 P = dict(zip(
3     users_ratings.index,
4     np.random.rand(len(users_ratings),
5                     number_LatentFactors).astype(np.float32)
6 ))
```

```
1 # Item-LF : 物品隐语义矩阵
2 Q = dict(zip(
3     items_ratings.index,
4     np.random.rand(len(items_ratings),
5                     number_LatentFactors).astype(np.float32)
6 ))
```

梯度下降训练

```
1 #梯度下降训练
2 for i in range(number_epochs):
3     print("iter%d"%i)
4     error_list = []
5     for uid, iid, r_ui in dataset.itertuples(index=False):
6         # User-LF P
7         ## Item-LF Q
8         v_pu = P[uid] #用户向量
9         v_qi = Q[iid] #物品向量
10        err = np.float32(r_ui - np.dot(v_pu, v_qi))
11
12        v_pu += alpha * (err * v_qi - reg_p * v_pu)
13        v_qi += alpha * (err * v_pu - reg_q * v_qi)
14
15        P[uid] = v_pu
16        Q[iid] = v_qi
17
18        error_list.append(err ** 2)
19    print(np.sqrt(np.mean(error_list)))
20
```

```
iter92
0.5749714601182931
iter93
0.574659329887967
iter94
0.5743535191226897
iter95
0.574053844275893
iter96
0.5737601184063581
iter97
0.5734721734647836
iter98
0.5731898348379139
iter99
0.5729129450689243
```

预测

```
1 #预测
2 uid = int(input("uid: "))
3 iid = int(input("iid: "))
```

```
uid: 1
iid: 1
```

```
1 if uid not in users_ratings.index or iid not in items_ratings.index:
2     print(globalMean)
3
4 p_u = P[uid]
5 q_i = Q[iid]
6 print(np.dot(p_u, q_i))
```

```
2.6548624
```


小结

■ **Traditional SVD**: 数学方式进行SVD矩阵分解, 必须稠密

■ **FunkSVD (LFM)** :
$$\underset{p_j, q_j}{\operatorname{argmin}} \sum_{(i,j) \in K} (m_{ij} - q_j^T p_i)^2 + \lambda(\|p_i\|_2^2 + \|q_j\|_2^2)$$

■ 梯度下降求解

■ **BiasSVD**:
$$\underbrace{\operatorname{arg min}}_{p_i, q_j} \sum_{i,j} (m_{ij} - \mu - b_i - b_j - q_j^T p_i)^2 + \lambda(\|p_i\|_2^2 + \|q_j\|_2^2 + \|b_i\|_2^2 + \|b_j\|_2^2)$$

■ 增加**用户偏置**和**物品偏置**

■ 本质都是利用**已有的评分**计算出**未知的评分**

The background features a light gray network graph with nodes and edges. Two large, semi-transparent spheres are positioned on the left and right sides, each containing a dense network of nodes and edges. The central text is white on a dark blue circular background.

1 4 chapter

Embedding

Embedding定义

- Embedding 字面理解是“嵌入”，实质是一种映射，从**语义空间**到**向量空间**的映射，同时尽可能在向量空间保持原样本在语义空间的关系。

颜色	One-hot表征	Embedding表征 (RGB)
茶花红	[1,0,0,0,0]	[0.933, 0.247, 0.302]
满江红	[0,1,0,0,0]	[0.655, 0.325, 0.353]
鹅冠红	[0,0,1,0,0]	[0.820, 0.102, 0.176]
高粱红	[0,0,0,1,0]	[0.753, 0.173, 0.220]
海棠红	[0,0,0,0,1]	[0.941, 0.216, 0.322]

色彩表征

Embedding原理

- 文本：“Embedding技术对深度学习推荐系统的重要性”
- 目标：将里面的每一个**分词**，都可以用**向量数字**表达
- 模型：Embedding有两种建模方式
CBOW（给定上下文，预测中心词）和
Skip-gram（给定输入词，预测上下文），这里选择Skip-gram作介绍

Embedding | 技术 | 对 | 深度学习 | 推荐系统 | 的 | 重要性



第一步：数据预处理

Embedding | 技术 | 对 | 深度学习 | 推荐系统 | 的 | 重要性

window1

window2

window3



第二步：滑窗获取素材

Word2vec模型的输入输出：

Sample1：技术	->	Embedding，深度学习
Sample2：深度学习	->	技术，推荐系统
Sample3：推荐系统	->	深度学习，重要性



第三步：数字向量表达对象

向量表达对象，技术：[...,...,...,...]

ALS原理

方法1：使用协同过滤矩阵分解的思想进行Embedding——ALS



评分矩阵	汤臣一品	上海康城	耀华路550弄
User1	4	3	?
User2	1	3	4

用户矩阵	豪华指数	刚需指数
User1	0.8	0.2
User2	0.2	0.8

物品矩阵	豪华指数	刚需指数
汤臣一品	5	0
上海康城	3	3
耀华路550弄	0	5

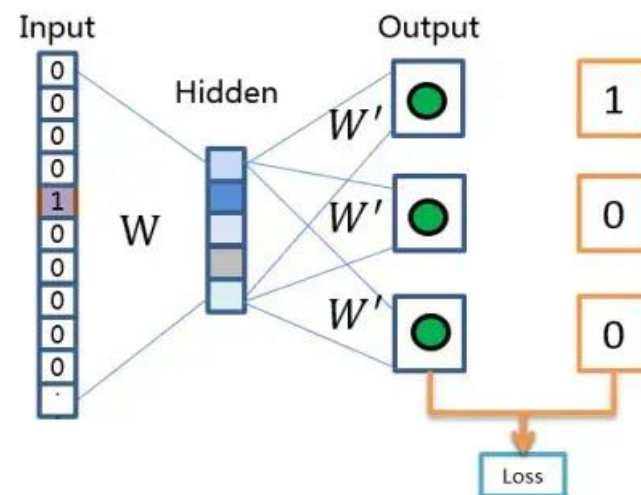
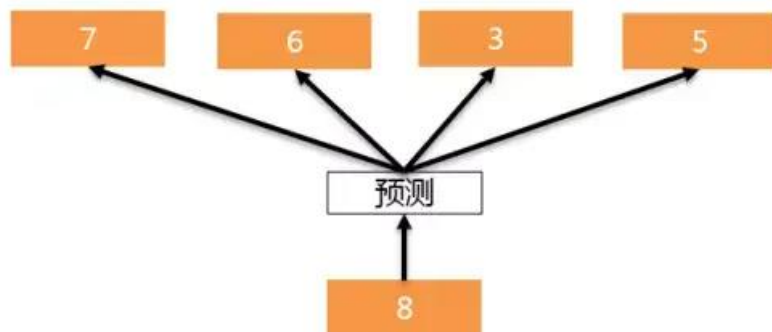
item2vec原理

- 参照word2vec方法，把item视为word，用户的**行为序列**视为一个集合，item间的**共现**为**正样本**，并按照item的频率分布进行负样本采样。
- 缺点就是没有建模用户对不同item的喜欢程度高低。

方法2：使用词嵌入思想进行Embedding——Skip-gram

用户浏览、关注、微聊、电话的房源序列

User1	8	7	5			
User2	3	5	2	7	8	3
User3	7	6	8	3	5	



$$\operatorname{argmax}_{\theta} \left(\sum_{(l,c) \in D_p} \log \frac{1}{1 + e^{-v'_c v_l}} + \sum_{(l,c) \in D_n} \log \frac{1}{1 + e^{v'_c v_l}} \right)$$

房源的相邻房源
作为正样本

随机抽取房源
作为负样本

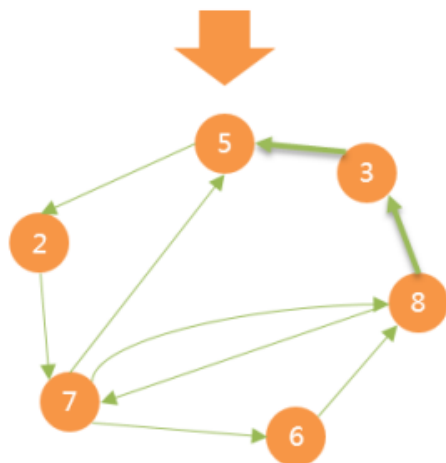
DeepWalk原理

- 将用户的item序列抽象成一个有向带权图，然后轮流以每个结点为出发点，根据权重进行随机游走，就可以形成新的item序列。然后对于新的item序列继续使用 Skip-gram 算法得到不同item的 Embedding 向量。

方法3：使用图的思想进行Embedding——DeepWalk

用户浏览、关注、微聊、电话的房源序列

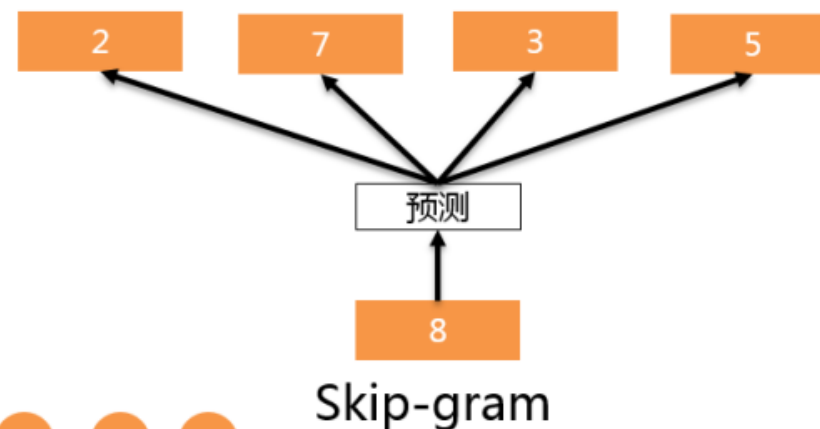
User1	8	7	5			
User2	3	5	2	7	8	3
User3	7	6	8	3	5	



用户行为加权图

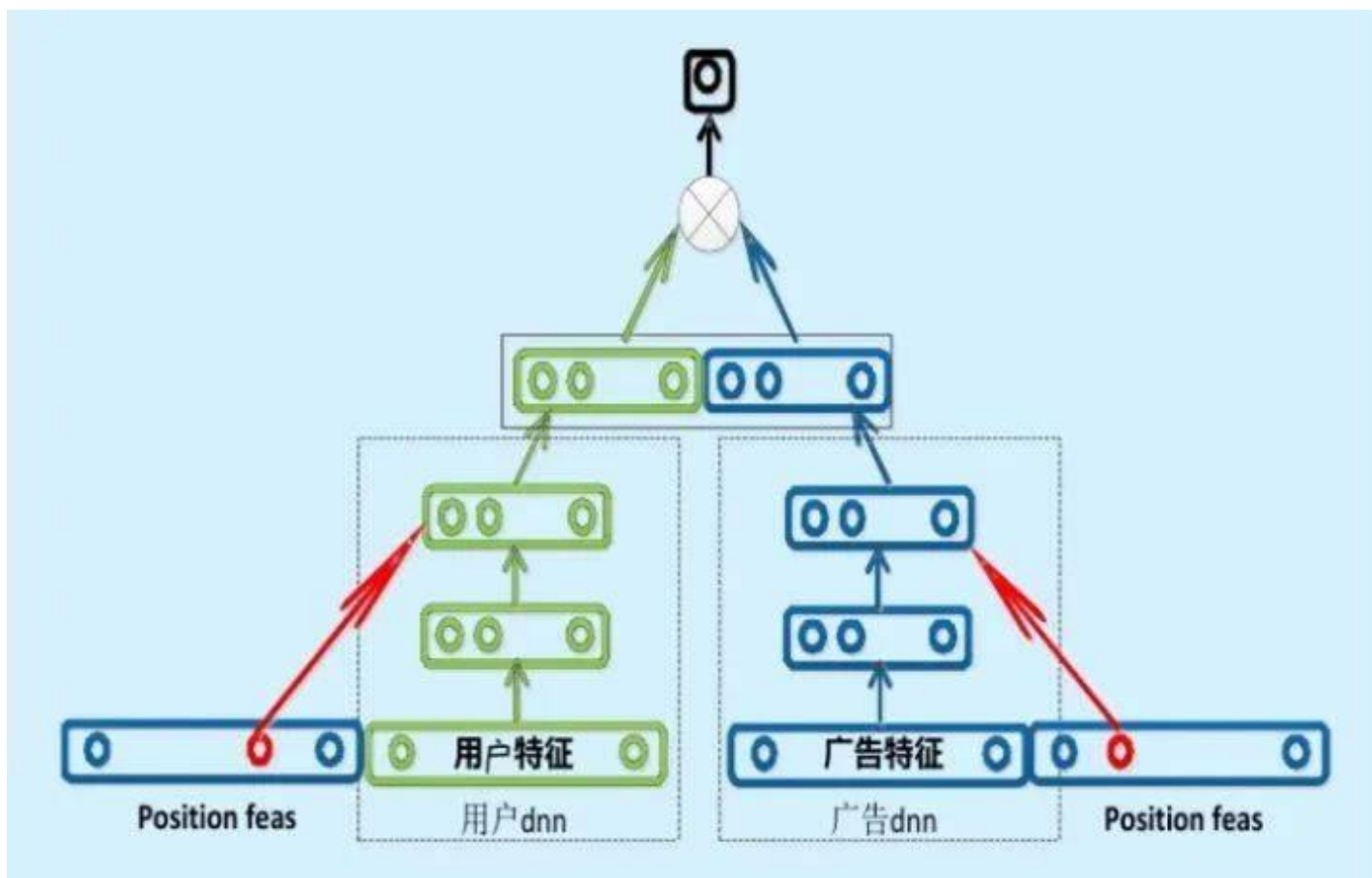


权重随机游走



双塔模型-"广义"的Item2Vec

- Embedding对物品进行向量化的方法远不止Item2Vec。广义上讲，任何能够生成物品向量的方法都可以被称为Item2Vec。典型的例子是曾在百度、Facebook等公司成功应用的双塔模型



实践步骤

■ 获取所有用户的点击序列

用户id	点击序列
A	1, 2, 3, 4
B	2, 3, 6, 7
C	3, 4, 7, 6

- 点击序列集合导入到item2vec中，得到每个物件的embedding
 - `model = word2vec.fit(click_order_str)`
- 通过物件集求embedding之和，得到人的embedding
- 以人的embedding为参数，得到相似度最高前N个物件
- 对相似集业务过滤，推荐给对应用户

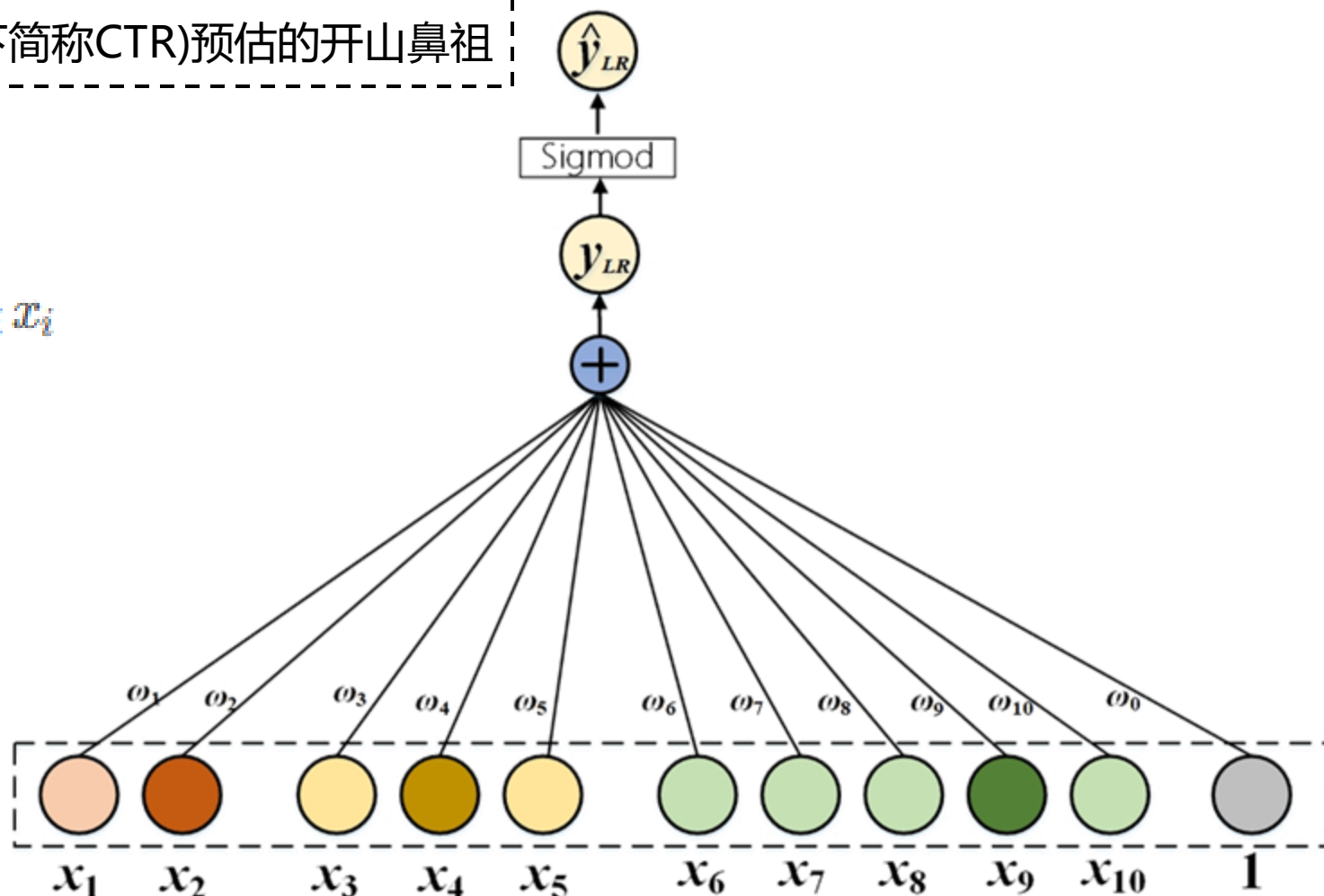
1
5
chapter

因子分解机(FM)家族

LR模型

- LR是点击率(click through rate,以下简称CTR)预估的开山鼻祖

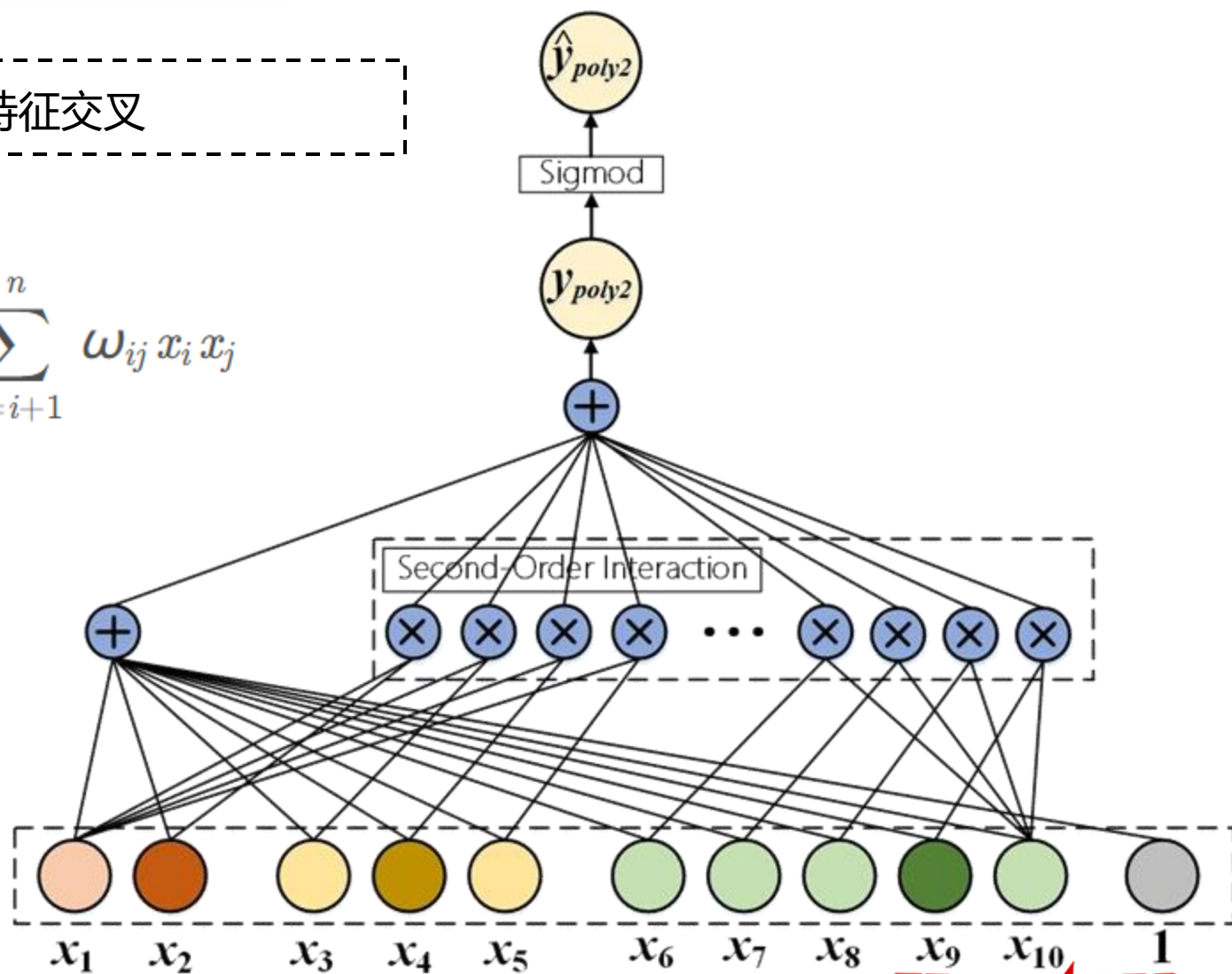
$$y_{LR} = \omega_0 + \sum_{i=1}^n \omega_i x_i$$



poly2

- 在LR基础上通过2阶多项式扩展添加二阶特征交叉

$$y_{poly2} = \omega_0 + \sum_{i=1}^n \omega_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \omega_{ij} x_i x_j$$

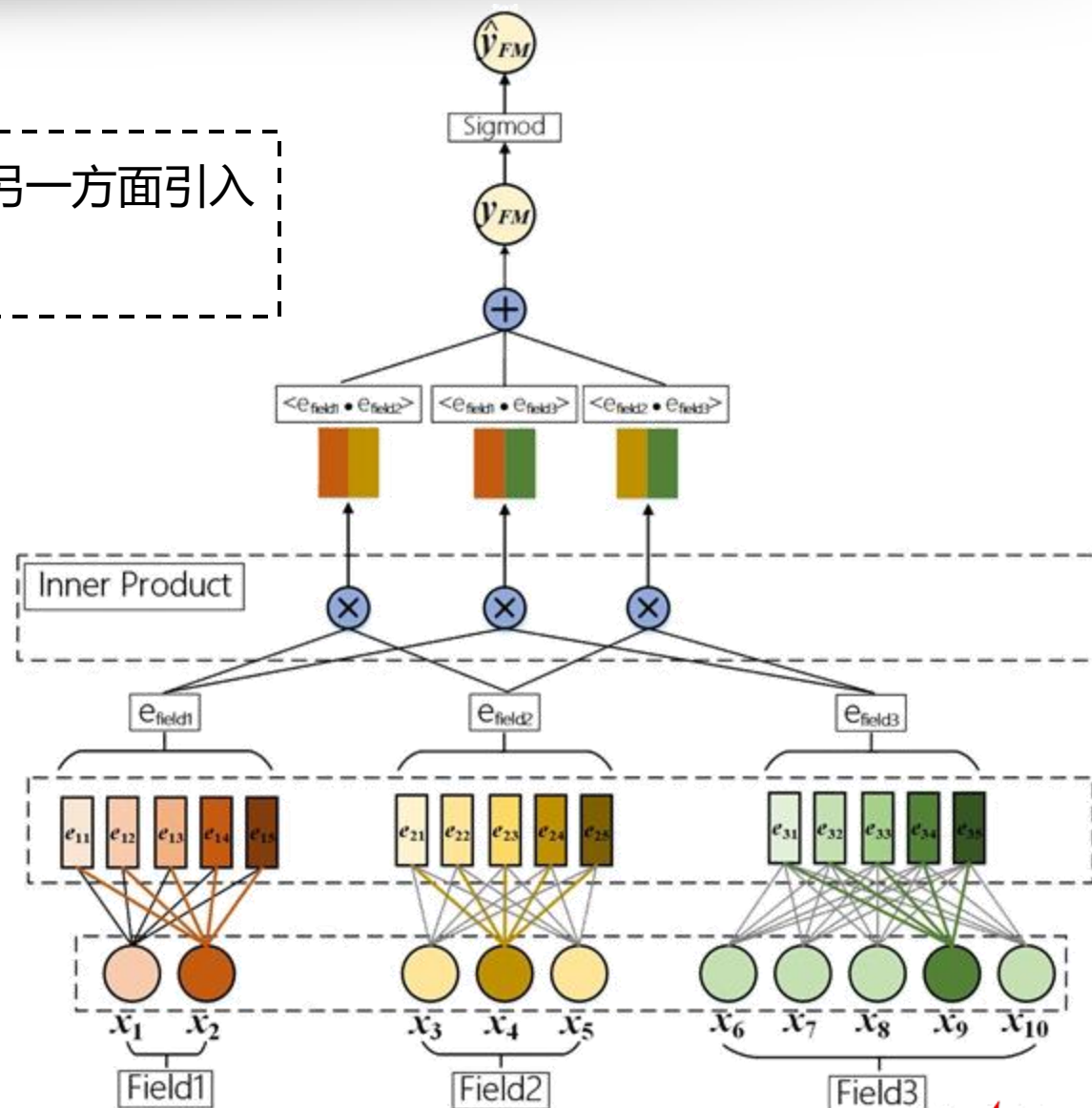


FM

- 将交互因子分解为两个特征向量权重的内积，在另一方面引入了embedding思想

$$y_{FM} = \omega_0 + \sum_{i=1}^n \omega_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \langle v_i \cdot v_j \rangle x_i x_j$$

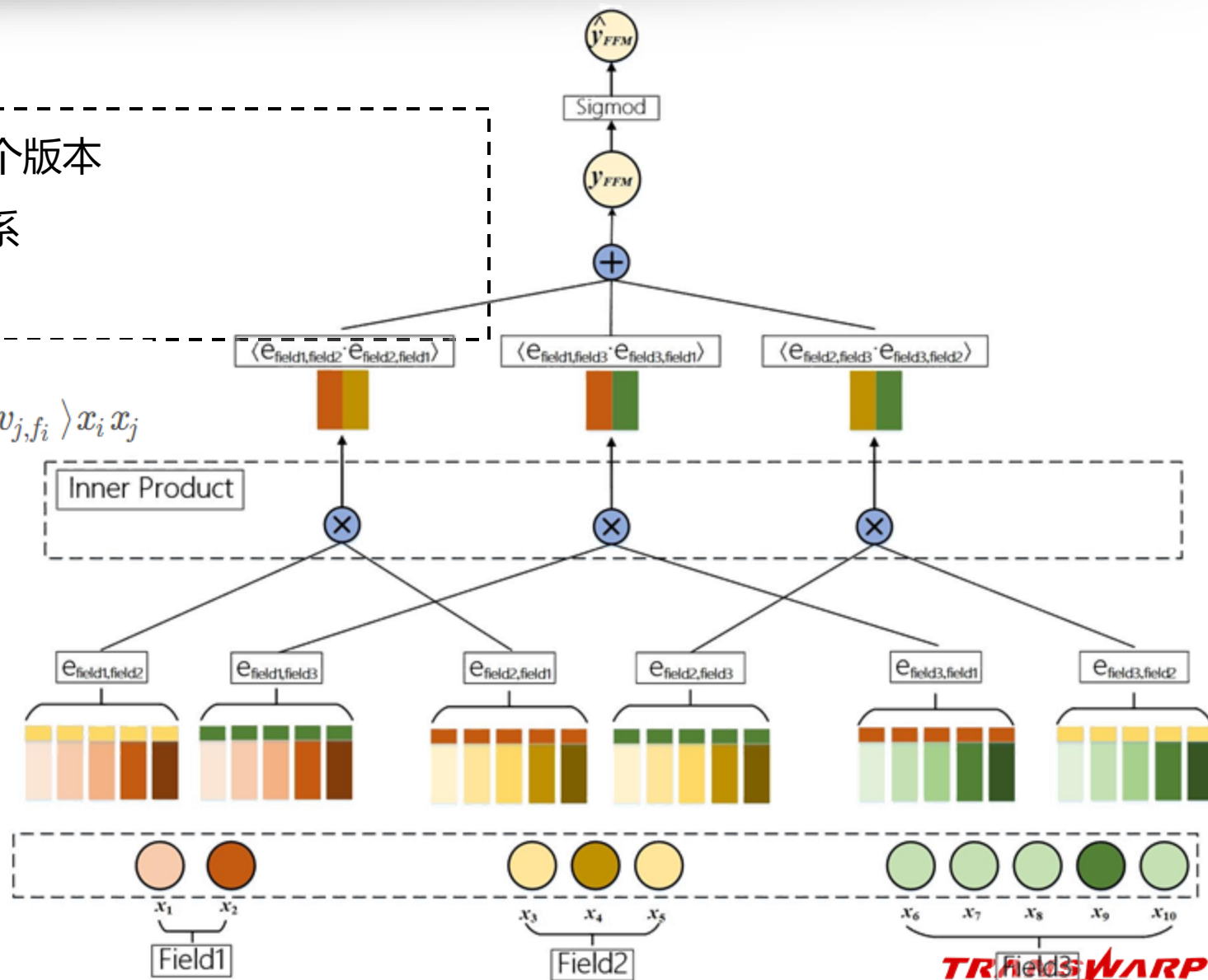
$$\sum_{i=1}^n \sum_{j=i+1}^n \langle v_i \cdot v_j \rangle x_i x_j = \frac{1}{2} \sum_{f=1}^k \left(\left(\sum_{i=1}^n v_{i,f} x_i \right)^2 - \sum_{i=1}^n v_{i,f}^2 x_i^2 \right)$$



FFM

- 一个特征的embedding向量有F-1个版本
- 更精细地刻画了特征直接交互的关系
- 参数量暴涨

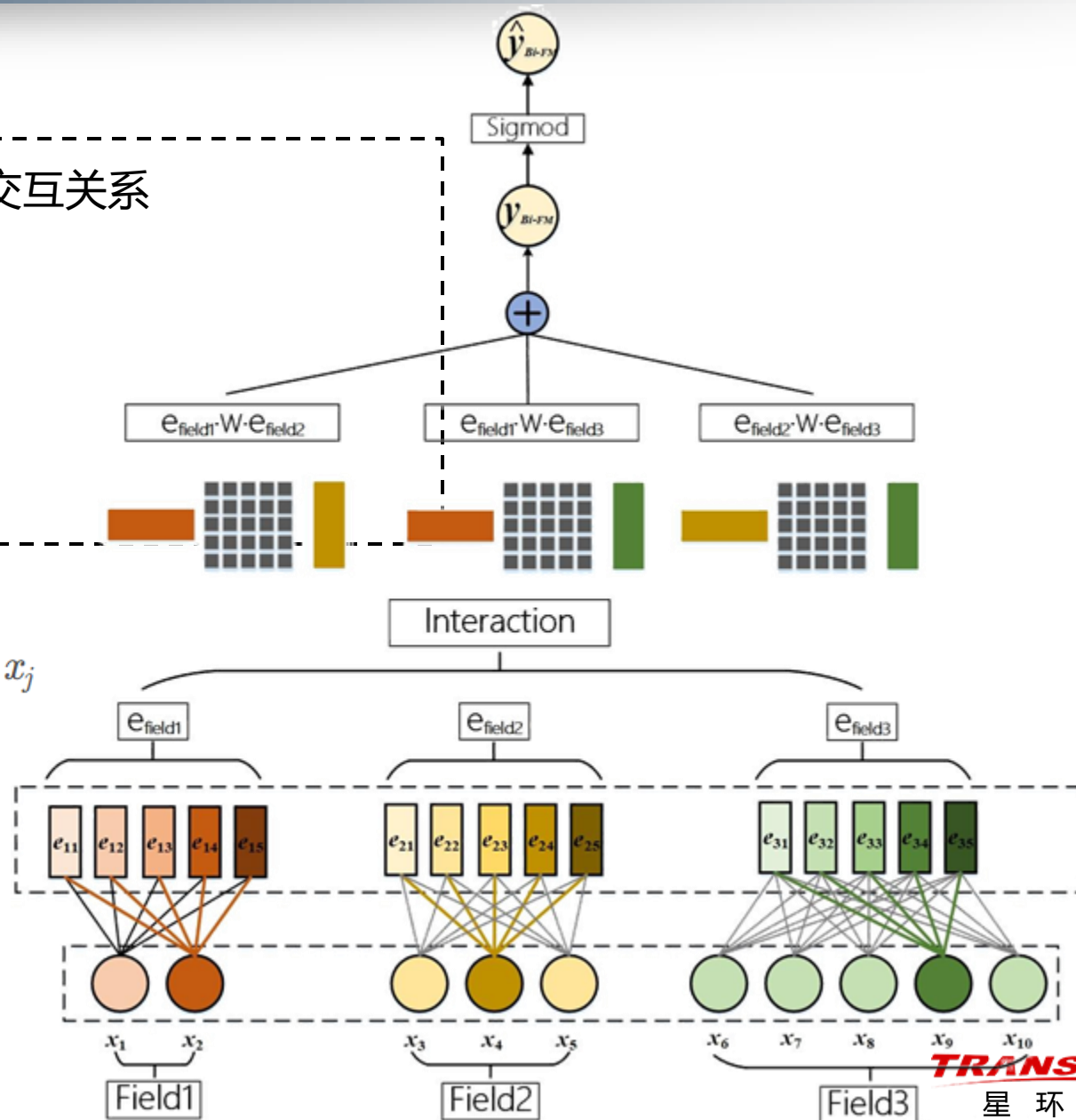
$$y_{FFM} = \omega_0 + \sum_{i=1}^n \omega_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \langle v_{i,f_j} \cdot v_{j,f_i} \rangle x_i x_j$$



Bilinear-FM

- 增加权重矩阵 W 来描述领域 i 和领域 j 交互关系
- W 可以有两种类型:
 - 1. 所有特征共享 W ;
 - 2. 一个领域组合一个 W_{ij}

$$y_{FFM} = \omega_0 + \sum_{i=1}^n \omega_i x_i + \sum_{i=1}^n \sum_{j=i+1}^n \langle v_{i,f_j} \cdot v_{j,f_i} \rangle x_i x_j$$



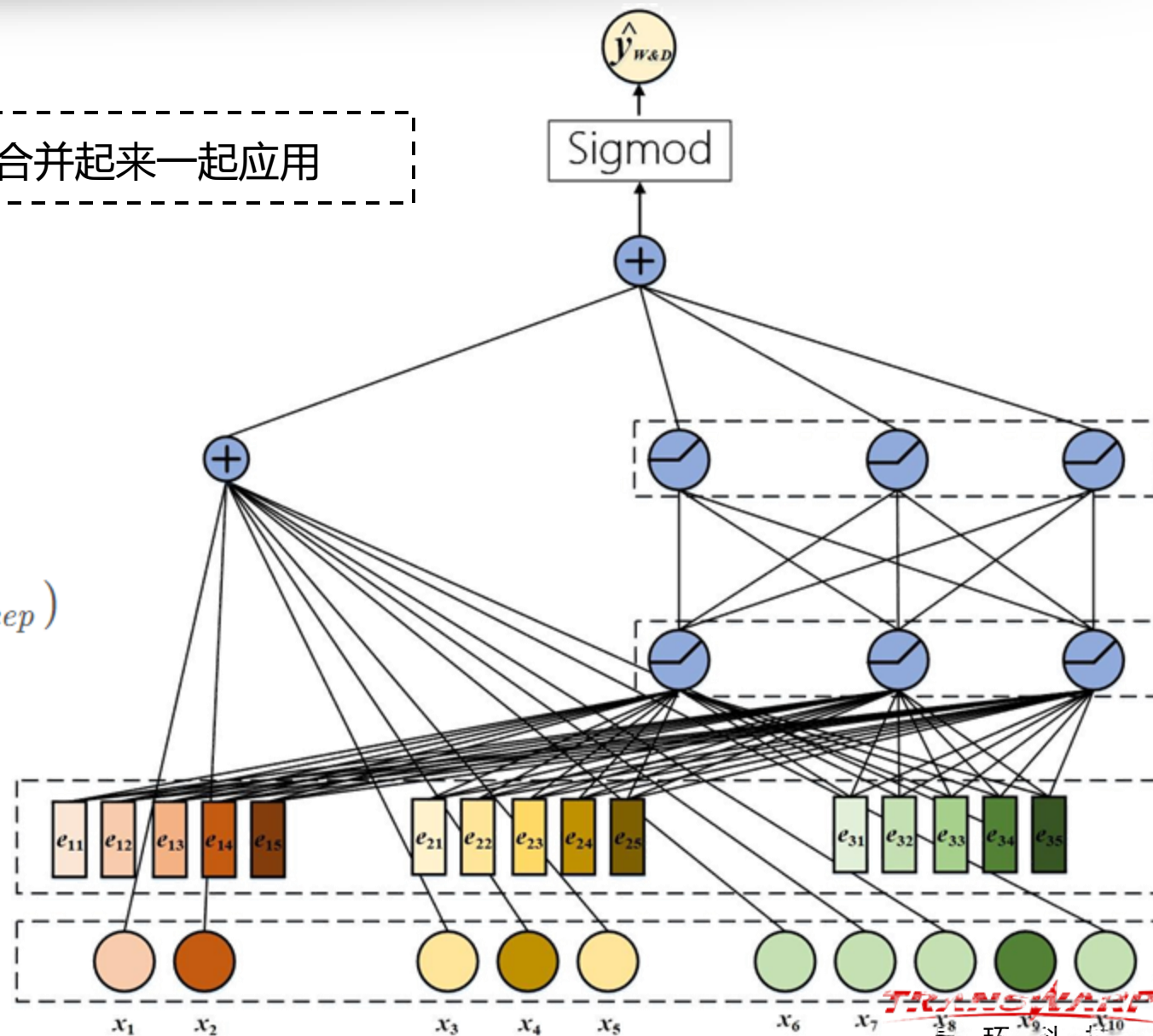
Wide&Deep

- 将浅层线性模型的LR与非线性模型DNN合并起来一起应用

$$y_{Wide} = \omega_0 + \sum_{i=1}^n \omega_i x_i$$

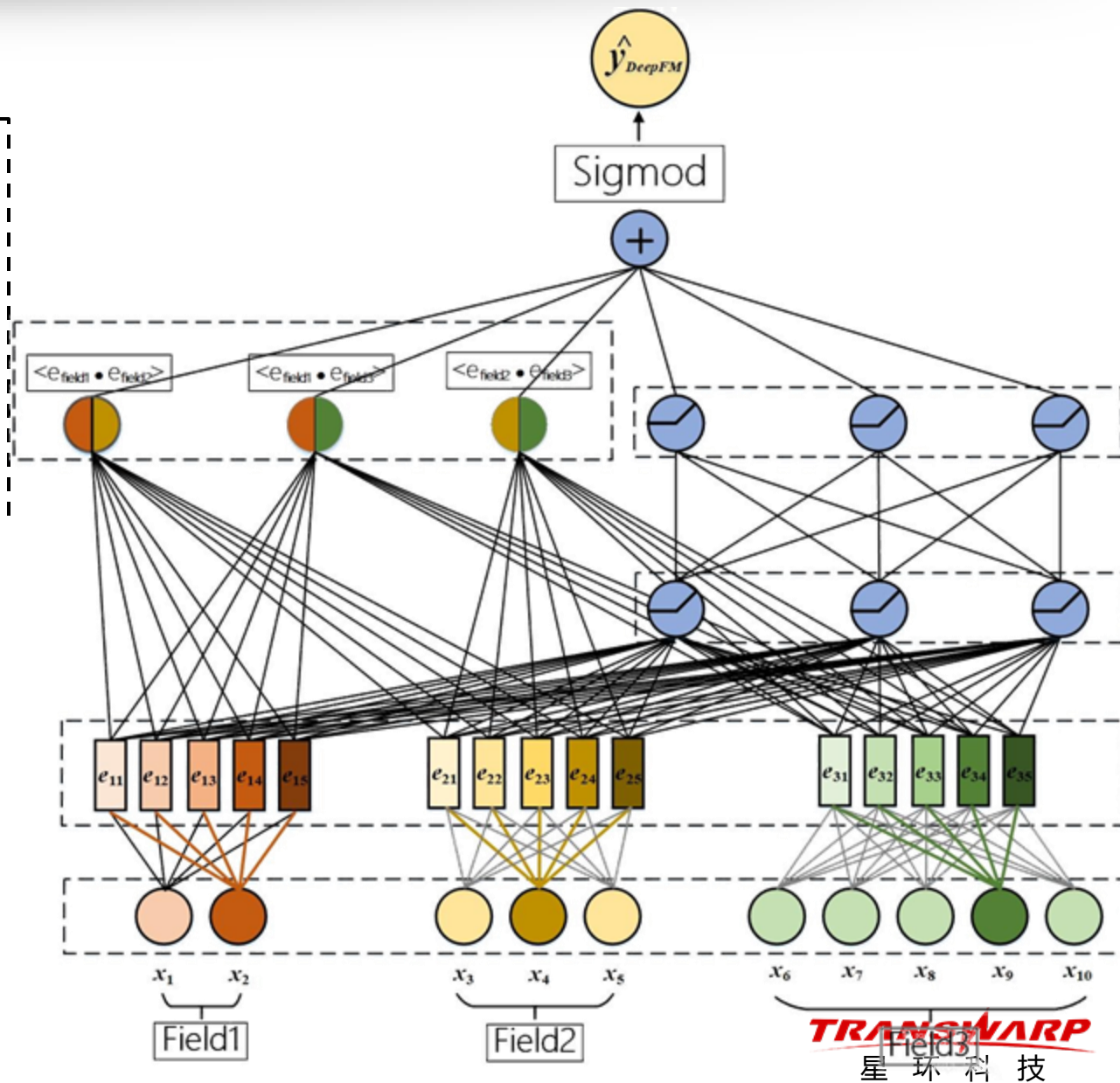
$$y_{Deep} = DNN(x_{embedding})$$

$$y_{W\&D} = \text{sigmoid}(W_{Wide} y_{Wide} + W_{Deep} y_{Deep})$$



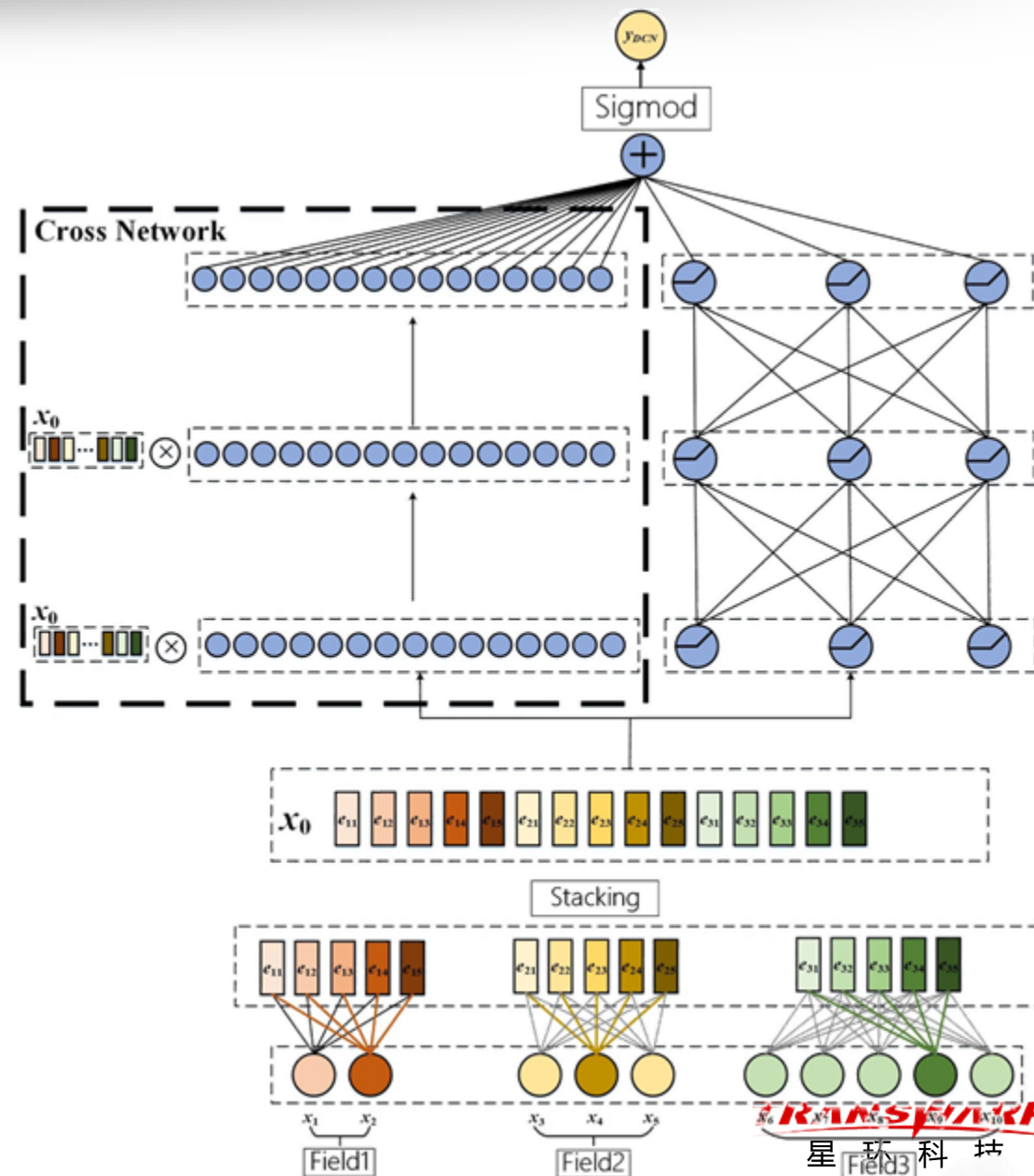
DeepFM

- DeepFM是在Wide&Deep基础上发展起来的，将Wide&Deep模型中的Wide部分替换为FM，并且使FM与DNN共享embedding输入值，这样就省去了原来Wide中的特征工程部分



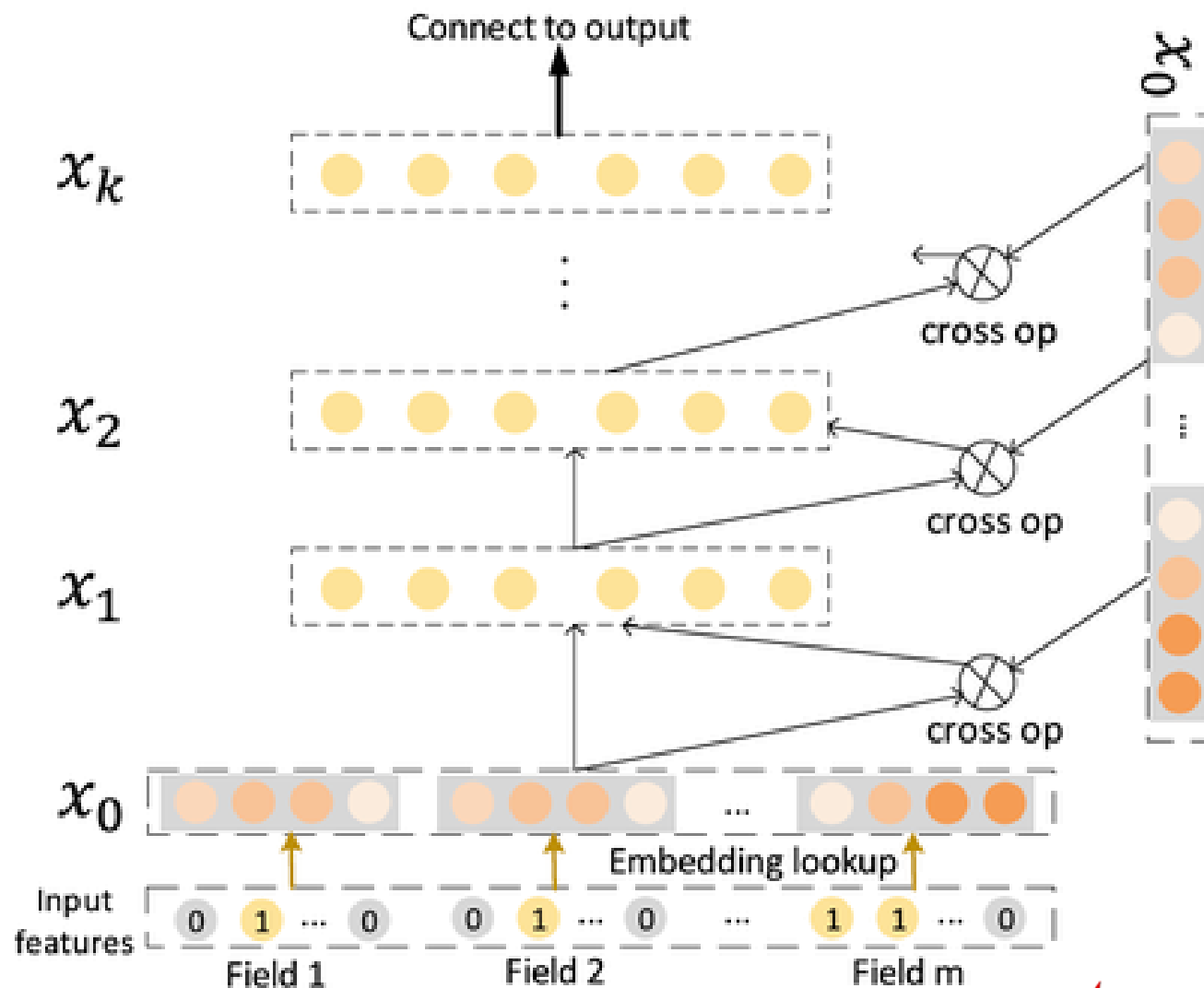
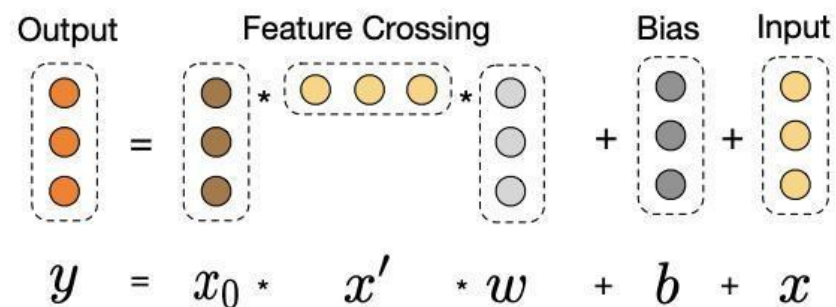
DCN

- DCN将DeepFM中的二阶特征组合部分升级为更高阶的代数多项式特征组合
- DCN将embedding后所有特征向量拼接成一个一维向量 x_0 ，再将 x_0 输入到Cross Network中以获得特征的多项式交叉。
- 通过改变Cross Network中的层数，特征交叉的多项式阶数可以人为设定。



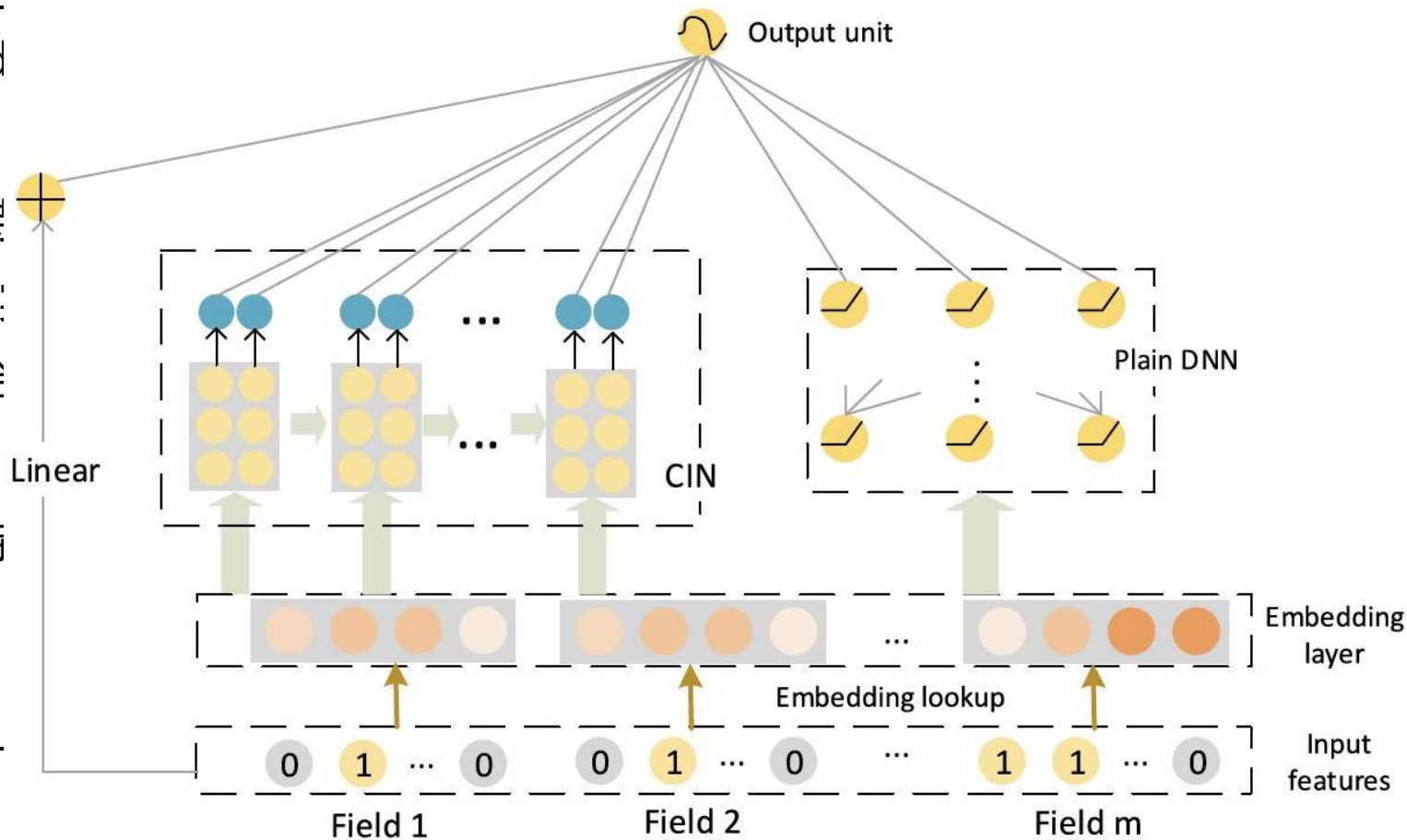
Cross Network单层中的特征高阶组合

$$x_{l+1} = x_0 x_l^T w_l + b_l + x_l$$



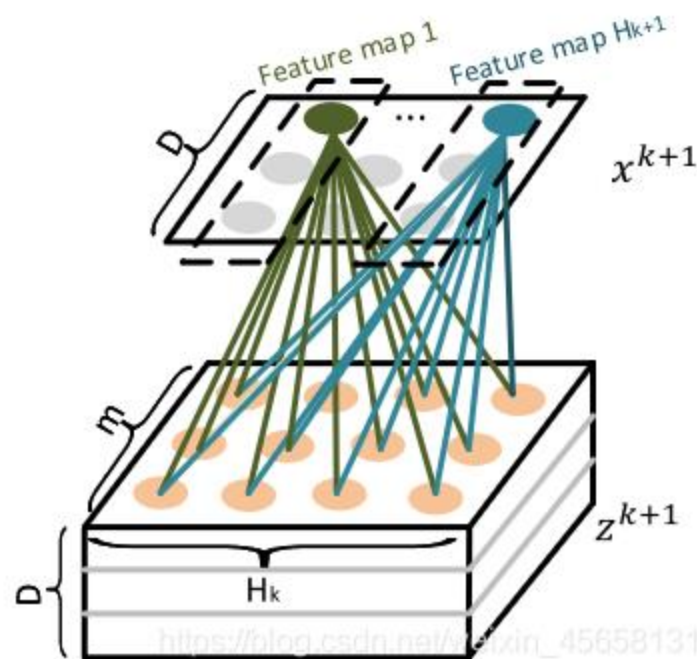
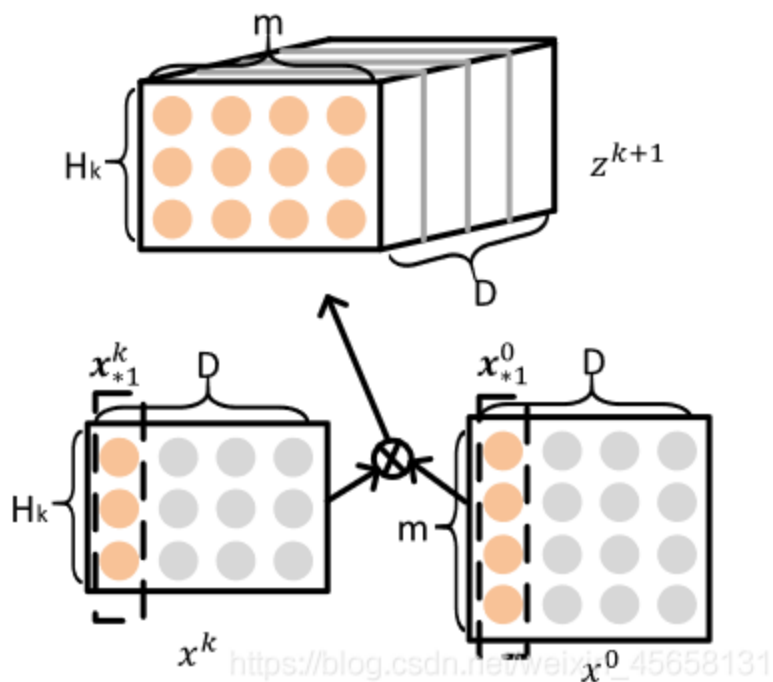
xDeepFM

- 与DCN相比, xDeepFM将embedd
拼接成矩阵形式, 在Compressed
Interaction Network(CIN)中实现特
的向量级别的多项式交叉组合(DCN:
特征向量元素级别的多项式交叉组合
看出xDeepFM更符合FM的本质
(embedding后的特征向量做二阶特
组合)。



CIN单层中的特征高阶组合

$$X_{h,*}^k = \sum_{i=1}^{H_{k-1}} \sum_{j=1}^m W_{ij}^{k,h} (X_{i,*}^{k-1} \circ X_{j,*}^0)$$



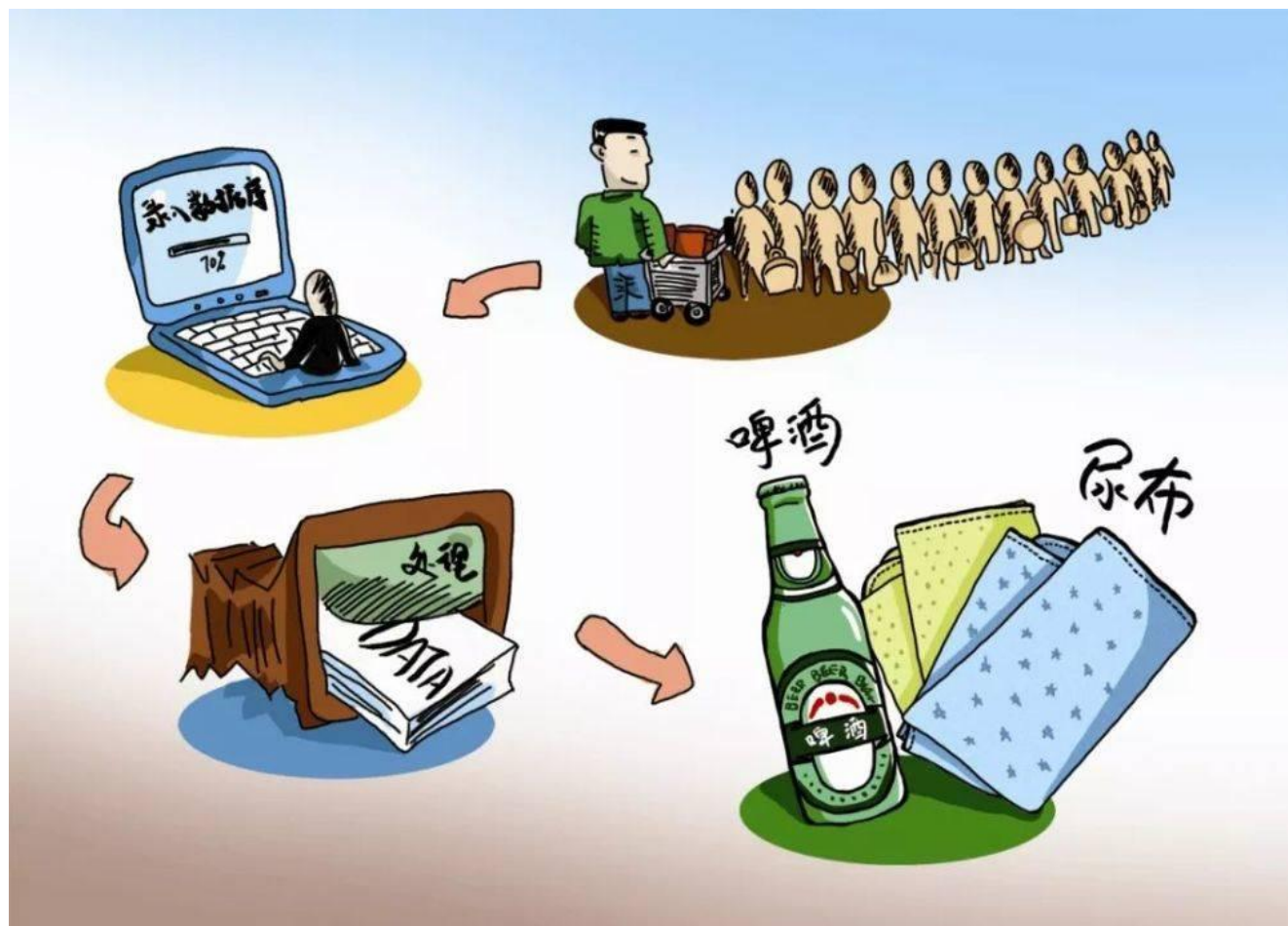
The background features a complex network diagram with numerous nodes (represented by small grey and black dots) connected by thin, light grey lines. These connections form a dense web that fills the entire frame, with some areas appearing more concentrated than others. The overall aesthetic is technical and modern, typical of data science or computer science presentations.

6 chapter

关联规则模型

关联规则挖掘

- 关联规则挖掘是数据挖掘中常用的手段，一般指的是从**交易数据库**、**关系数据库**以及**其他的数据集中**发现项或对象的**频繁的模式**(frequent patterns)、**关联**(association)的过程。此方法一般用于在**购物篮分析**(market basket analysis)中。最早是用于发现超市销售数据库中不同商品之间的关联关系，最经典的莫过于**啤酒与尿布**的这个段子。
- 主要算法：**Apriori**、**FP-growth**



Apriori算法

- 支持度 (Support) : $\text{support} = P(AB)$, 指的是**事件A和事件B同时发生**的概率 (联合概率)。

$$\text{support}(X, Y) = P(X, Y) = \frac{\text{number}(X, Y)}{\text{number}(\text{all sample})}$$

- 置信度 (confidence) : $\text{confidence} = P(B|A) = P(AB)/P(A)$, 指的是发生**事件A**的**基础上发生事件B**的概率(条件概率)

$$\text{confidence}(X \Rightarrow Y) = P(Y|X) = \frac{P(X, Y)}{P(X)} = \frac{\text{number}(X, Y)}{\text{number}(X)}$$

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

Example:

$\{\text{Milk, Diaper}\} \Rightarrow \{\text{Beer}\}$

$$s = \frac{\sigma(\text{Milk, Diaper, Beer})}{|T|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk, Diaper, Beer})}{\sigma(\text{Milk, Diaper})} = \frac{2}{3} = 0.67$$

Apriori算法流程

1. 发现所有的频繁集: 支持度 $\geq \text{minsupport}$ 的所有项集
 1. 统计每个k项候选集的支持度, 找出频繁的k项集: L_k
 2. 利用频繁的k项集生成k+1项候选集: C_{k+1} , $k=k+1$
2. 将每个频繁集生成可能的关联规则

1. 扫描TDB, 对每个候选项计算支持度得到表 C_1
2. 比较 C_1 中各个后选项支持度与最小支持度, 得到1维的最大项集 L_1
3. 由 L_1 得到候选集 C_2
4. 由 C_2 比较支持度得到 L_2
5. 由 L_2 得到 C_3
6. 由 C_3 得到3维项集 L_3

minsup = 2

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

1st scan

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

L_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

C_2

Itemset	sup
{A, B}	1
{A, C}	2
{A, E}	1
{B, C}	2
{B, E}	3
{C, E}	2

2nd scan

Itemset	sup
{A, B}	1
{A, C}	2
{A, E}	1
{B, C}	2
{B, E}	3
{C, E}	2

L_2

Itemset	sup
{A, C}	2
{B, C}	2
{B, E}	3
{C, E}	2

C_3

Itemset	sup
{B, C, E}	2

3rd scan

Itemset	sup
{B, C, E}	2

L_3

Apriori算法-生成关联规则

- 现有频繁项集 l ，生成每个非空子集 s ，若 s 满足最小置信度：

$$confidence((l - s) \rightarrow s) = \frac{sup(l)}{sup(l - s)} \geq minconf$$

- 则输出关联规则 $(l-s) \rightarrow s$

L_1

itemset	sup.
{A}	2
{B}	3
{C}	3
{E}	3

L_2

itemset	sup
{A C}	2
{B C}	2
{B E}	3
{C E}	2

L_3

itemset	sup
{B C E}	2

- $minconf=80\%$

❖ For {BCE}:
 Confidence(BE → C) = 2/3 < 80%,
 Confidence(BC → E) = 2/2 > 80%
 Confidence(CE → B) = 2/2 > 80%
 Confidence(B → CE) = 2/3 < 80%
 Confidence(C → BE) = 2/3 < 80%
 Confidence(E → BC) = 2/3 < 80%

FP-growth简介

- 一种非常好的发现频繁项集算法。
- 它基于Apriori算法构建，将数据集存储在叫做 FP树 的数据结构中。
- FP-growth只能高效地发现频繁项集，不能用于发现关联规则。
- Fp-growth只需对数据进行两次扫描，因此比Apriori速度快
 - 基于数据构建FP树
 - 从FP树种挖掘频繁项集

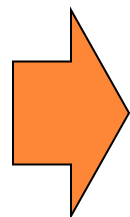
FP-growth原理: 1) 基于数据构建FP树

ID	Items
1	牛奶, 鸡蛋, 面包, 薯片
2	鸡蛋, 爆米花, 薯片, 啤酒
3	牛奶, 面包, 啤酒
4	牛奶, 鸡蛋, 面包, 爆米花, 薯片, 啤酒
5	鸡蛋, 面包, 薯片
6	鸡蛋, 面包, 啤酒
7	牛奶, 面包, 薯片
8	牛奶, 鸡蛋, 面包, 黄油, 薯片
9	牛奶, 鸡蛋, 黄油, 薯片

- 遍历所有的数据集合, 计算所有项的支持度。
- 丢弃非频繁的项。
- 基于 支持度 降序排序所有的项。
- 所有数据集合按照得到的顺序重新整理。
- 重新整理完成后, 丢弃每个集合末尾非频繁的项。

FP-growth原理: 1) 基于数据构建FP树

ID	Items
1	牛奶, 鸡蛋, 面包, 薯片
2	鸡蛋, 爆米花, 薯片, 啤酒
3	牛奶, 面包, 啤酒
4	牛奶, 鸡蛋, 面包, 爆米花, 薯片, 啤酒
5	鸡蛋, 面包, 薯片
6	鸡蛋, 面包, 啤酒
7	牛奶, 面包, 薯片
8	牛奶, 鸡蛋, 面包, 黄油, 薯片
9	牛奶, 鸡蛋, 黄油, 薯片



ID	Items
1	薯片, 鸡蛋, 面包, 牛奶
2	薯片, 鸡蛋, 啤酒
3	面包, 牛奶, 啤酒
4	薯片, 鸡蛋, 面包, 牛奶, 啤酒
5	薯片, 鸡蛋, 面包
6	鸡蛋, 面包, 啤酒
7	薯片, 面包, 牛奶
8	薯片, 鸡蛋, 面包, 牛奶
9	薯片, 鸡蛋, 牛奶

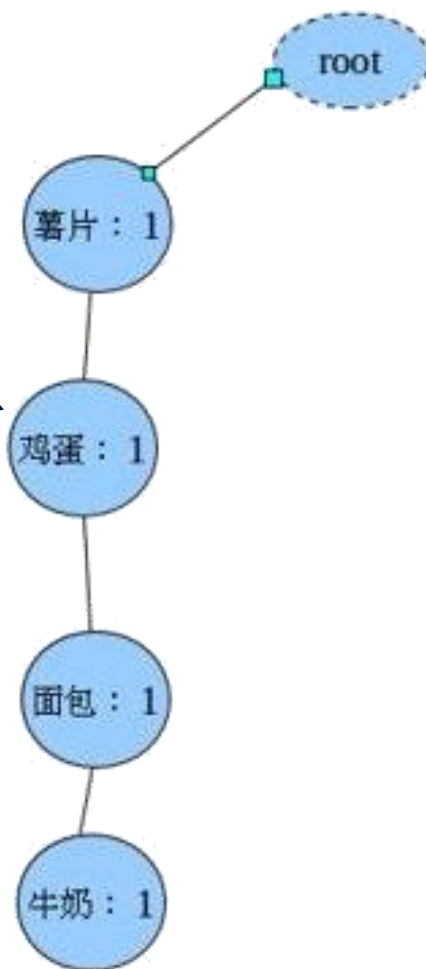
- step1: 扫描数据库, 每项商品按频数递减排序, 删除频数小于最小支持度 (minsup=4) 的商品
- 薯片: 7, 鸡蛋: 7, 面包: 7, 牛奶: 6, 啤酒: 4

- step2: 对于每一条记录按照【薯片: 7, 鸡蛋: 7, 面包: 7, 牛奶: 6, 啤酒: 4】排序

FP-growth原理: 1) 基于数据构建FP树

- step3: 把step2重排序的记录, 插入到fp-tree种
- step3.1: 插入第一条 (第一步有一个虚的根节点)

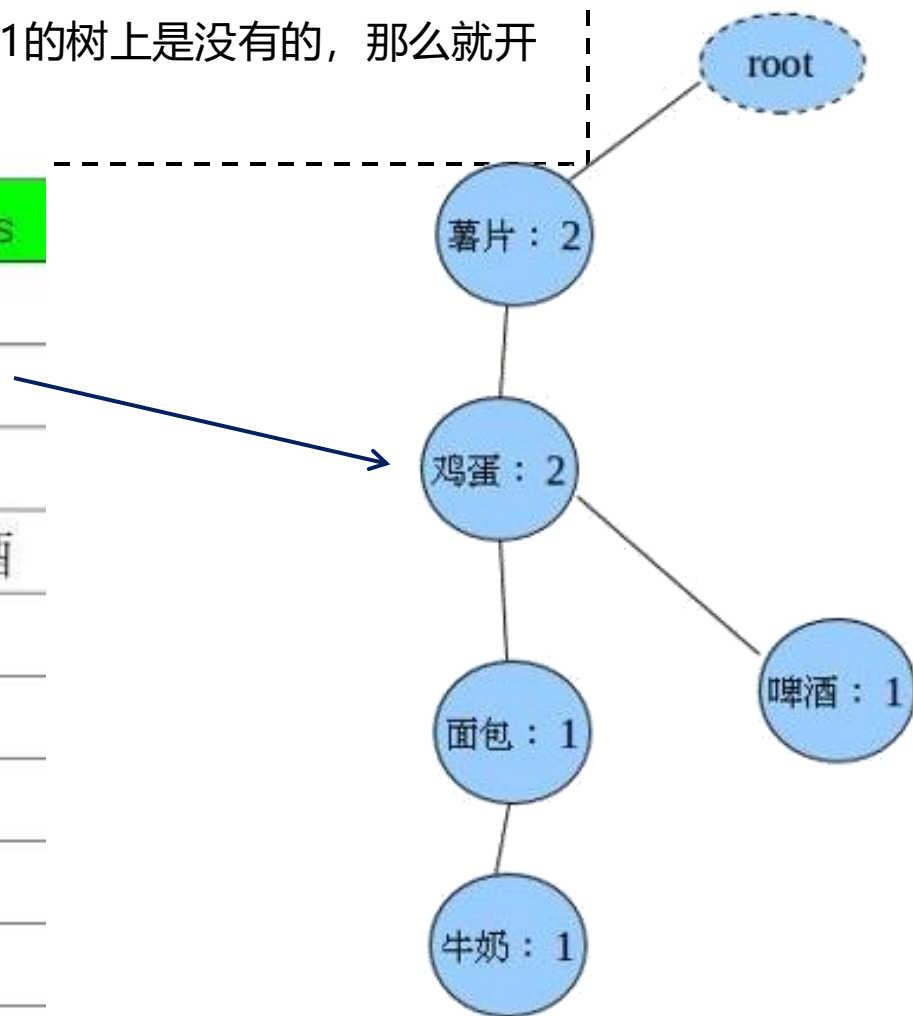
ID	Items
1	薯片, 鸡蛋, 面包, 牛奶
2	薯片, 鸡蛋, 啤酒
3	面包, 牛奶, 啤酒
4	薯片, 鸡蛋, 面包, 牛奶, 啤酒
5	薯片, 鸡蛋, 面包
6	鸡蛋, 面包, 啤酒
7	薯片, 面包, 牛奶
8	薯片, 鸡蛋, 面包, 牛奶
9	薯片, 鸡蛋, 牛奶



FP-growth原理: 1) 基于数据构建FP树

- Step3.2 : 插入第二条。根结点不管, 然后插入薯片, 在step3.1的基础上+1, 则记为2;同理鸡蛋记为2;啤酒在step3.1的树上是没有的, 那么就开一个分支。

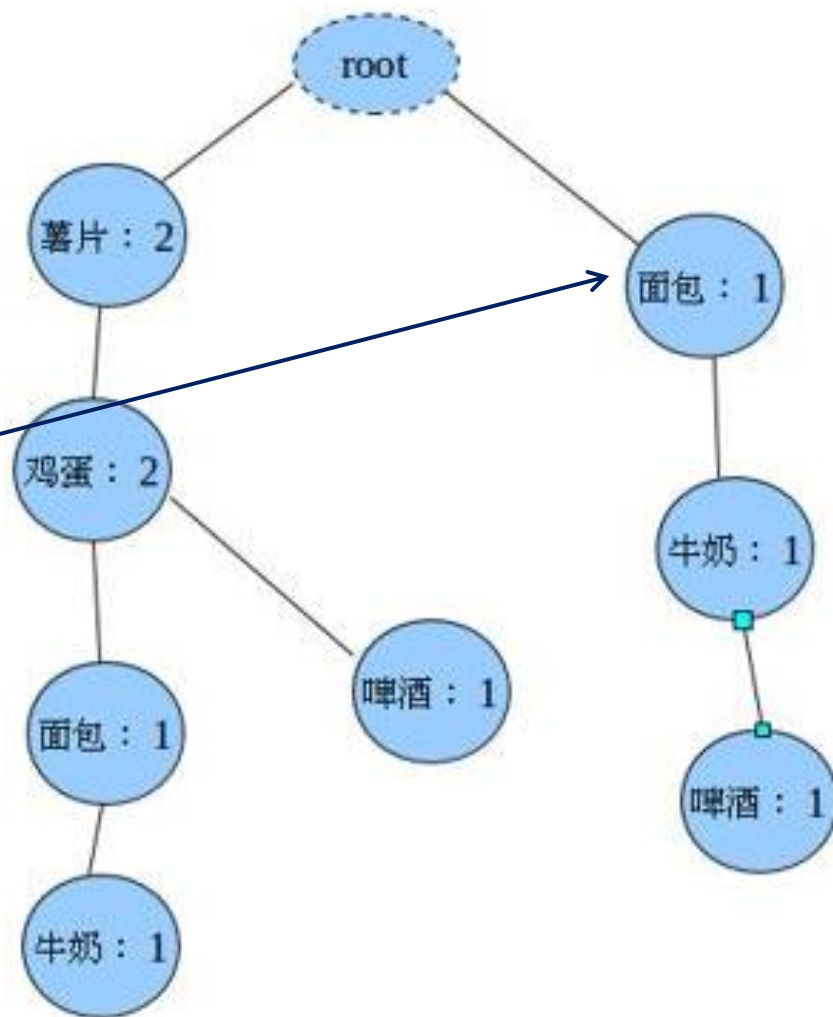
ID	Items
1	薯片, 鸡蛋, 面包, 牛奶
2	薯片, 鸡蛋, 啤酒
3	面包, 牛奶, 啤酒
4	薯片, 鸡蛋, 面包, 牛奶, 啤酒
5	薯片, 鸡蛋, 面包
6	鸡蛋, 面包, 啤酒
7	薯片, 面包, 牛奶
8	薯片, 鸡蛋, 面包, 牛奶
9	薯片, 鸡蛋, 牛奶



FP-growth原理: 1) 基于数据构建FP树

■ Step3.3: 插入第三条

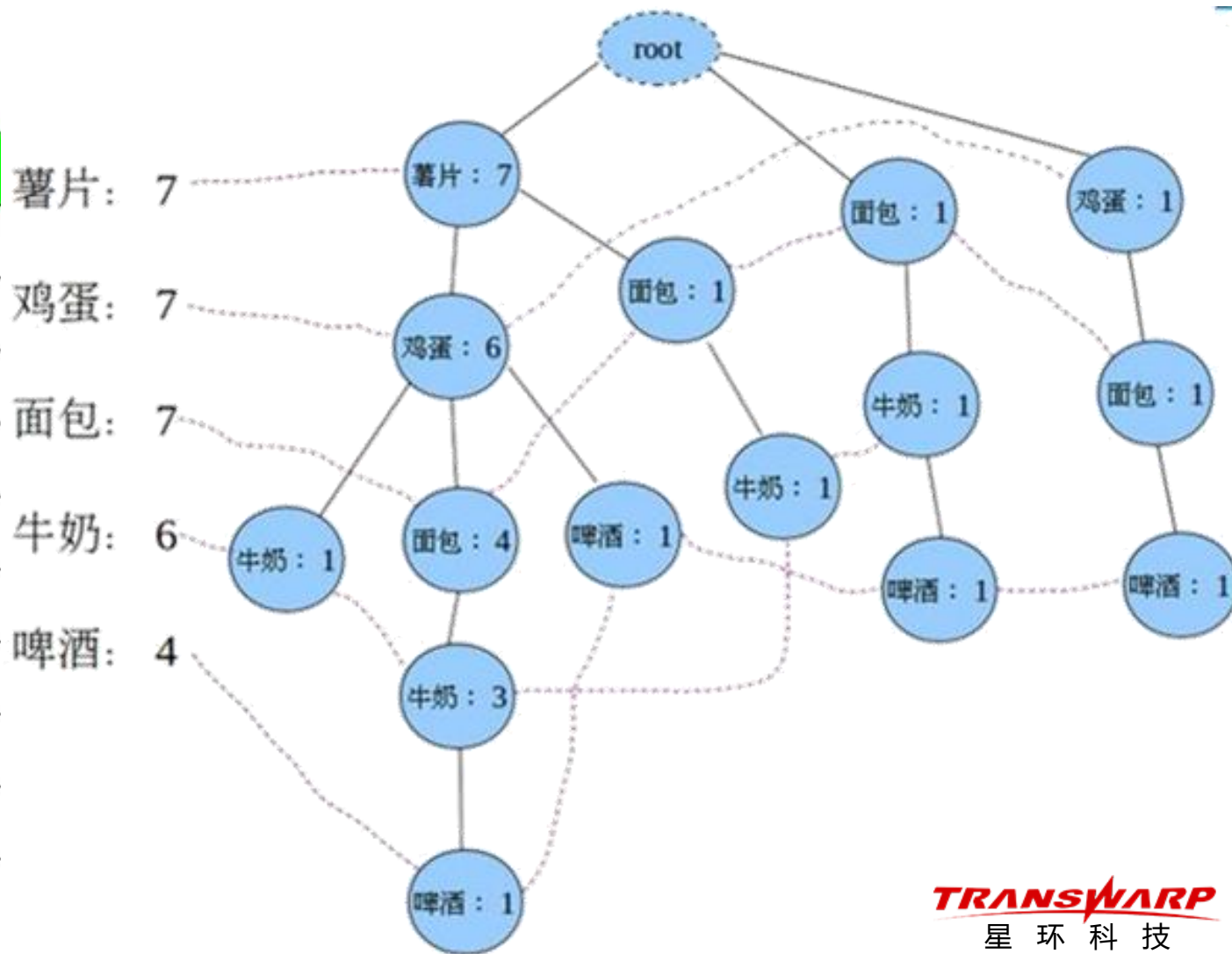
ID	Items
1	薯片, 鸡蛋, 面包, 牛奶
2	薯片, 鸡蛋, 啤酒
3	面包, 牛奶, 啤酒
4	薯片, 鸡蛋, 面包, 牛奶, 啤酒
5	薯片, 鸡蛋, 面包
6	鸡蛋, 面包, 啤酒
7	薯片, 面包, 牛奶
8	薯片, 鸡蛋, 面包, 牛奶
9	薯片, 鸡蛋, 牛奶



FP-growth原理: 1) 基于数据构建FP树

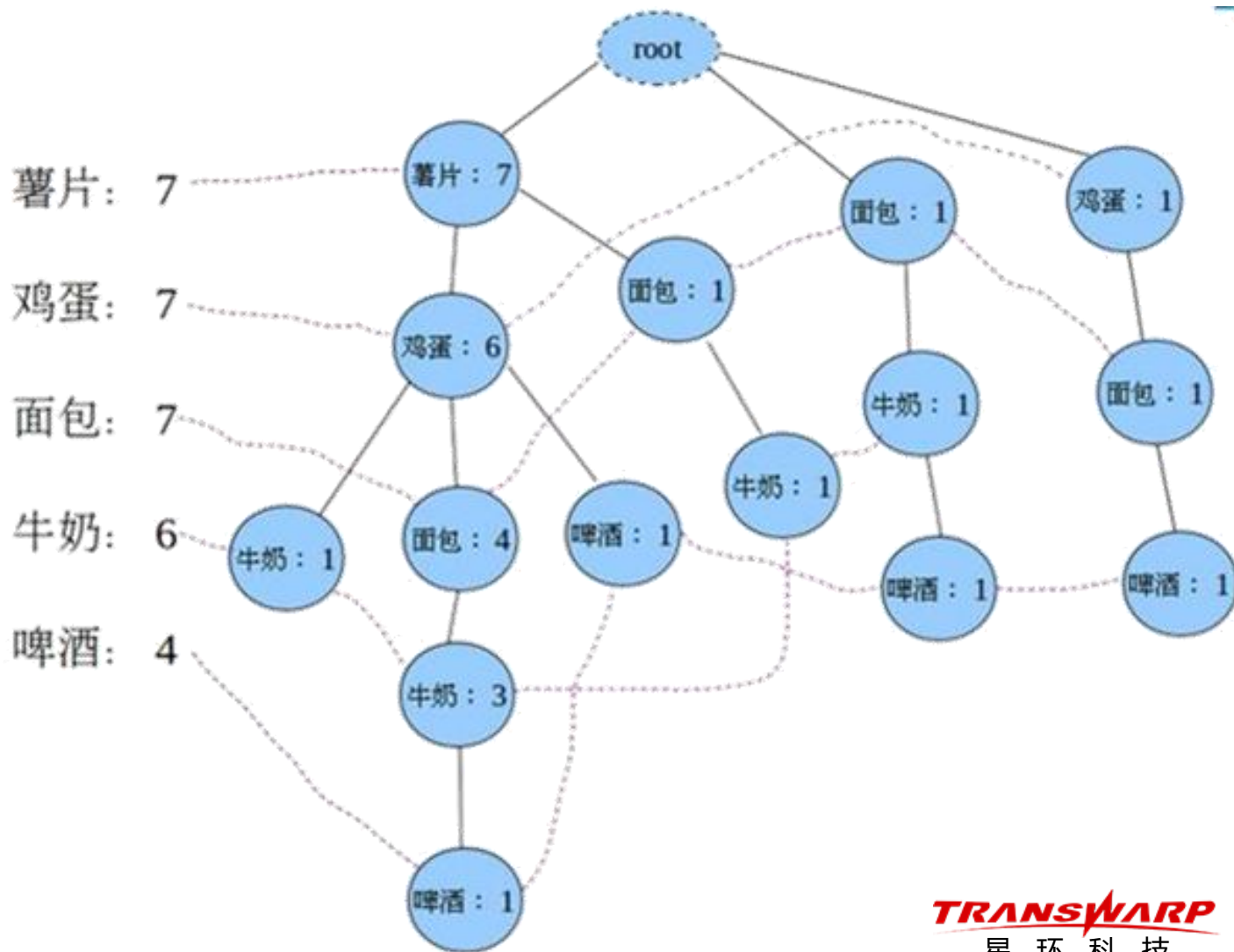
- 同理, 剩余记录依次插入fp-tree中

ID	Items
1	薯片, 鸡蛋, 面包, 牛奶
2	薯片, 鸡蛋, 啤酒
3	面包, 牛奶, 啤酒
4	薯片, 鸡蛋, 面包, 牛奶, 啤酒
5	薯片, 鸡蛋, 面包
6	鸡蛋, 面包, 啤酒
7	薯片, 面包, 牛奶
8	薯片, 鸡蛋, 面包, 牛奶
9	薯片, 鸡蛋, 牛奶



FP-growth原理： 1) 基于数据构建FP树

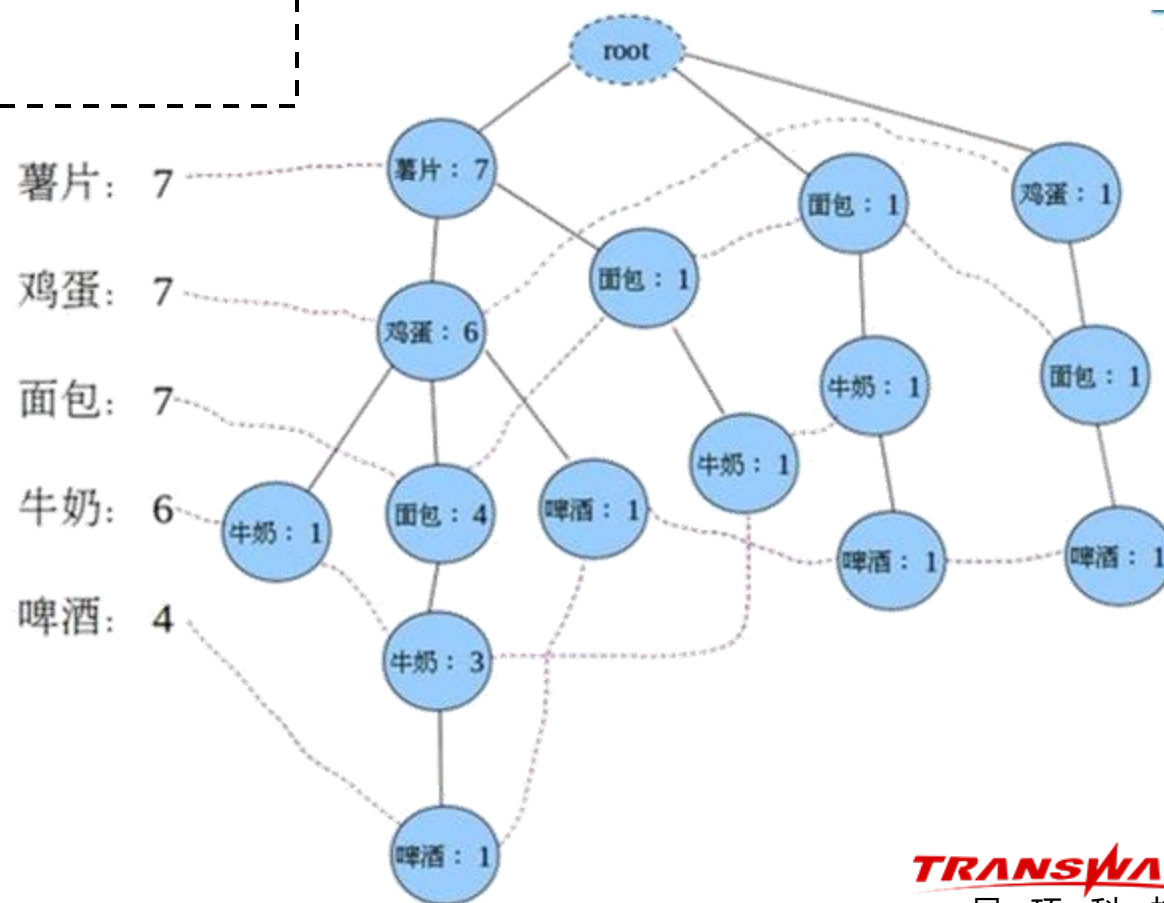
- 图中左边的一列叫做头指针表，树中相同名称的节点要链接起来，链表的第一个元素就是头指针表里的元素。
- 虚线连接起来的表示同一个商品，各个连接的数字加起来就是该商品出现的总次数。



FP-growth原理：2) 从FP树中挖掘出频繁项集

- 遍历表头项中的每一项（以"牛奶：6"为例），从FP-Tree中找到所有的"牛奶"结点，向上遍历它的祖先结点，得到4条路径，如表所示。

ID	Items
1	薯片: 7, 鸡蛋: 6, 牛奶: 1
2	薯片: 7, 鸡蛋: 6, 面包: 4, 牛奶: 3
3	薯片: 7, 面包: 1, 牛奶: 1
4	面包: 1, 牛奶: 1



FP-growth原理: 2) 从FP树中挖掘出频繁项集

- 对于每一条路径上的节点，其count都设置为牛奶的count（路径中最末尾的商品数）

ID	Items
1	薯片: 7, 鸡蛋: 6, 牛奶: 1
2	薯片: 7, 鸡蛋: 6, 面包: 4, 牛奶: 3
3	薯片: 7, 面包: 1, 牛奶: 1
4	面包: 1, 牛奶: 1



ID	Items
1	薯片: 1, 鸡蛋: 1, 牛奶: 1
2	薯片: 3, 鸡蛋: 3, 面包: 3, 牛奶: 3
3	薯片: 1, 面包: 1, 牛奶: 1
4	面包: 1, 牛奶: 1

FP-growth原理: 2) 从FP树中挖掘出频繁项集

- 因为每一项末尾都是牛奶，可以把牛奶去掉，得到条件模式基，此时的后缀模式是：牛奶。

ID	Items
1	薯片: 1, 鸡蛋: 1, 牛奶: 1
2	薯片: 3, 鸡蛋: 3, 面包: 3, 牛奶: 3
3	薯片: 1, 面包: 1, 牛奶: 1
4	面包: 1, 牛奶: 1



ID	Items
1	薯片: 1, 鸡蛋: 1
2	薯片: 3, 鸡蛋: 3, 面包: 3
3	薯片: 1, 面包: 1
4	面包: 1

导入库和数据

```
1 # 导入库
2 import pandas as pd
3 import apriori
4 #加载数据文件
5 data = pd.read_csv('data/order_table.csv')
```

```
1 data.head()
```

	order_id	product_name
0	201901010000001	citrus fruit
1	201901010000001	semi-finished bread
2	201901010000001	margarine
3	201901010000001	ready soups
4	201901010000002	tropical fruit

转换为关联所用的记录模式

```
1 # 转换为关联所用的记录模式
2 order_ids = pd.unique(data['order_id'])
3 order_records = [data[data['order_id']==each_id]['product_name'].tolist() for each_id in order_ids]
4 order_records
```

```
[['citrus fruit', 'semi-finished bread', 'margarine', 'ready soups'],
 ['tropical fruit', 'yogurt', 'coffee'],
 ['whole milk'],
 ['pip fruit', 'yogurt', 'cream cheese ', 'meat spreads'],
 ['other vegetables',
 'whole milk',
 'condensed milk',
 'long life bakery product'],
 ['whole milk', 'butter', 'yogurt', 'rice', 'abrasive cleaner'],
 ['milk /butter']]
```

挖掘频繁项

```
1 # 通过调用自定义的apriori做关联分析
2 minS = 0.1 # 定义最小支持度阈值
3 minC = 0.5 # 定义最小置信度阈值
4 L, suppData = apriori.apriori(order_records, minSupport=minS) # 计算得到满足最小支持度的规则
5 L
```

```
[[frozenset({'shopping bags'}),
  frozenset({'root vegetables'}),
  frozenset({'soda'}),
  frozenset({'bottled water'}),
  frozenset({'rolls/buns'}),
  frozenset({'other vegetables'}),
  frozenset({'whole milk'}),
  frozenset({'yogurt'}),
  frozenset({'tropical fruit'})],
[]]
```

挖掘关联规则

```
1 rules = apriori.generateRules(order_records, L, suppData, minConf=minC) # 计算满足最小置信度的规则  
2 rules#因为最小支持度和最小置信度太大了
```

```
[]
```


调整参数

```
1 #调整参数
2 minS = 0.02 # 定义最小支持度阈值
3 minC = 0.2 # 定义最小置信度阈值
4 L, suppData = apriori.apriori(order_records, minSupport=minS) # 计算得到满足最小支持度的规则
5 L
```

```
[[frozenset({'meat'}),
  frozenset({'sliced cheese'}),
  frozenset({'onions'}),
  frozenset({'frozen meals'})],
```

```
1 rules = apriori.generateRules(order_records, L, suppData, minConf=minC) # 计算满足最小置信度的规则
2 rules
```

```
[(frozenset({'other vegetables'}),
  frozenset({'yogurt'}),
  104,
  0.0464,
  0.2391,
  1.722),
  (frozenset({'yogurt'}),
  frozenset({'other vegetables'}),
```



Q&A

TRANSWARP
星环科技